

Parallel Numerical Library

One numerical library, seven execution backends,
and an honest account of what each one costs

Olajide Badejo

August 2026

Abstract

This report describes a C++20 numerical library in which twelve iterative solvers are written once, against a single problem interface, and run unchanged over seven execution backends: serial, OpenMP, POSIX threads, a `std::jthread` pool, MPI, hybrid MPI with threads, and CUDA. The design makes identical numerics across backends a tested invariant rather than an assumption, and in its strong form: with the deterministic reduction mode every shared memory backend at every worker count produces bit identical iterates, and the GPU sweeps are bit identical to the CPU too.

That invariant is what makes the measurements comparable. Because every backend computes the same values, the differences between them are attributable to the execution model and to nothing else.

The convergence behaviour of the solver zoo is checked against closed form theory rather than asserted: the Jacobi spectral radius against $\cos(\pi h)$, the ratio of two between Jacobi and Gauss Seidel iteration counts, Young's optimal relaxation factor as a genuine minimum, and the change of growth order from $O(n^2)$ to $O(n)$ that optimal relaxation buys.

The CPU versus GPU comparison is constructed so that the hardware does not decide the argument. Iteration counts, which are hardware free, are reported first. Efficiency against each device's own measured STREAM bandwidth is reported second. Absolute time is reported last and labelled as a property of one particular pair of devices. The resulting speedup is then decomposed into the part that is the memory system and the part that is the implementation.

Contents

1	Introduction	4
1.1	The question	4
1.2	The invariant that makes it work	4
1.3	The solver zoo, and why it is a family	5
1.4	A comparison the hardware does not decide	5
1.5	What the environment allowed, and what it did not	5
1.6	How to read this report	6
2	Mathematical background	7
2.1	The splitting framework	7
2.2	The model problem	7
2.3	Richardson iteration	8
2.4	Jacobi	8
2.5	Gauss Seidel, and the factor of two	8
2.6	Successive over relaxation	9
2.7	Red black ordering	9
2.8	Block methods	10
2.9	Conjugate gradient	10
2.10	The numerics module	11
3	Parallelisation design	12
3.1	One interface	12
3.1.1	Why the callbacks are chunk level	12
3.1.2	What backends may not do	12
3.2	Determinism as a design decision	13
3.3	Contraction, and a claim that nearly was not true	13
3.4	What each backend does underneath	14
3.4.1	Shared memory	14
3.4.2	Distributed memory	14
3.4.3	Sequential dependence across ranks	14
3.5	The CUDA boundary	15

3.6	Thread pinning under a hypervisor	15
4	Comparison methodology	16
4.1	Why the naive comparison is misleading	16
4.2	The protocol	16
4.2.1	First: iterations to tolerance	16
4.2.2	Like against like	17
4.2.3	Second: bytes per unknown, counted	17
4.2.4	Pre registered: the rule that settles the traffic model	17
4.2.5	Third: each device's own bandwidth, measured	18
4.2.6	The comparable number	18
4.2.7	Last: absolute time	19
4.3	Measuring at sizes where methods do not converge	19
4.4	Experimental hygiene	19
4.5	Comparing these numbers with release 1.0.0's	20
4.6	What this comparison does not establish	20
5	Results	22
5.1	The machine	22
5.2	Iterations to tolerance	22
5.2.1	Agreement with theory	22
5.3	Backend cost on identical work	26
5.4	Scaling and the core knee	28
5.4.1	What could and could not be established	28
5.4.2	The knee is not where the objective expected it	30
5.5	Pinning	31
5.6	The cost of determinism	34
5.7	Scheduling	35
5.8	Distributed scaling and the communication fraction	36
5.9	The CPU versus GPU comparison	36
5.9.1	Decomposing the speedup	36
5.9.2	The crossover, derived	39
6	Discussion	40
6.1	What the equivalence invariant bought	40
6.2	The methods that do not parallelise, and why they are still here	40
6.3	Where the environment set the limits	41
6.4	Things that were wrong before they were right	41
6.5	What the traffic model changes, and what it does not	42
6.6	Related work	43

6.7	What a reader should take away	44
7	Conclusion	45
7.1	What was built	45
7.2	What was established	45
7.3	What was not established	46
7.4	Further work	46

Chapter 1

Introduction

1.1 The question

The question worth asking about parallel programming models is not whether OpenMP is fast. It is what each model costs on identical work. That question is hard to answer honestly because the usual comparison changes two things at once: a study that reimplements an algorithm per model ends up measuring the care that went into each reimplementation, and one that compares different algorithms on different devices ends up measuring the algorithms.

This project answers it by construction. One numerical library, twelve iterative solvers written exactly once against a single problem interface, and seven execution backends behind a single parallelism interface. A solver does not know which backend it is running on. A backend does not know which solver it is running. What varies between measurements is therefore the execution model, and that is enforced by a test rather than promised in a paragraph.

1.2 The invariant that makes it work

The design commits to something stronger than the usual claim of numerical agreement. With the deterministic reduction mode:

- every shared memory backend, at every worker count, produces *bit identical* iterates, not merely close ones;
- the GPU sweeps produce bit identical values to the CPU as well;
- across MPI rank counts, results agree to reduction tolerance, and the reason they cannot be bit identical is documented rather than hidden.

This is achievable because floating point addition is not associative and the library therefore refuses to let the association vary. The chunk grid used by every reduction is fixed by the problem size alone and never by the worker count, so the partial sums are always combined in the same order regardless of which worker produced which one or when. Chapter 3 explains the mechanism; the equivalence suite asserts exact equality across twelve solvers, two problem families, four backends and seven worker counts.

Getting there required disabling fused multiply add contraction on both the host and the device. The SOR update has the shape $ab + cd$, which a compiler is free to fuse, and the host was fusing it while the device was not. A claim of identical numerics that holds only when the compiler

happens not to contract is a much weaker claim than it appears, so contraction is off on both sides. On kernels that are bandwidth bound this costs nothing measurable.

1.3 The solver zoo, and why it is a family

Nine methods are required by the objectives and three more earn their place. Eleven of the twelve are one family: every classical stationary method is a choice of matrix splitting $A = M - N$, and the differences between Richardson, Jacobi, the Gauss Seidel variants, SOR and the block methods are entirely differences in the choice of M . Chapter 2 presents them that way, derives each in a few lines from the common framework, and cites the source for each convergence result.

Conjugate gradient is the twelfth and is deliberately not in that list. It is not stationary, its optimality argument is different in kind, and presenting it as the contrast is what makes the family visible as a family.

Every convergence claim in this report is checked as a measurement. The Jacobi spectral radius is compared against $\cos(\pi h)$; the ratio of Jacobi to Gauss Seidel iteration counts is compared against the factor of two that Young's theory predicts; the closed form optimal relaxation factor is tested as a minimum by perturbing it in both directions; and the change of growth order from $O(n^2)$ to $O(n)$ is demonstrated by measuring how the counts grow as the grid doubles.

1.4 A comparison the hardware does not decide

A five point stencil sweep performs three memory operations and about six floating point operations per unknown. It is bandwidth bound on every machine ever built. A ratio of runtimes between a CPU and a GPU on such a kernel is therefore, to within a small correction, a ratio of memory bandwidths wearing the costume of a statement about parallel programming.

Chapter 4 sets out the protocol that avoids this. Iteration counts come first because they are properties of the mathematics and transfer to any machine. Efficiency against each device's own measured STREAM triad bandwidth comes second, because it is dimensionless and says how well a device is being used rather than which device is larger. Absolute time comes last, labelled as a property of this specific pair of devices. The measured speedup is then decomposed into the factor that is the memory system and the factor that is the implementation, so a reader can see which is which.

Like is compared against like throughout. Natural ordering Gauss Seidel cannot run on a GPU at all, because it is sequentially dependent; the comparison therefore pairs GPU Jacobi against CPU Jacobi and GPU red black Gauss Seidel against CPU red black Gauss Seidel, and reports the cost of the red black reordering separately so that it is charged to both sides equally.

1.5 What the environment allowed, and what it did not

The target machine has an Intel Core i7-14700K with eight performance cores and twelve efficiency cores, and everything runs inside WSL2. The guest does not pass the heterogeneity through: it reports fourteen uniform cores, identical cache sizes on every logical processor, and no hybrid feature flag. Thread affinity binds to a guest virtual processor that the hypervisor may place on any host core.

Rather than assume a mapping that cannot be verified from inside the guest, the library measures. A probe times an identical kernel on each logical processor and reports a two group

split only when the throughputs separate by a margin that dominates the noise. The core knee that the objectives ask for is taken from the aggregate scaling curve, which depends only on how many cores are engaged and not on which, and therefore survives virtualisation. Section 5.4 reports both what could and what could not be established.

1.6 How to read this report

Chapter 2 derives the solver family and states the convergence theory. Chapter 3 describes the backend interface, what each implementation does underneath, and the communication analysis for the distributed backends. Chapter 4 is the comparison protocol, written to be difficult to misquote. Chapter 5 contains the measurements. Chapter 6 interprets them, including the parts that did not come out as expected. Chapter 7 summarises what the work established and what it did not.

Every number in Chapter 5 is generated from `experiments/results/summary.csv` by a script, and every row of that file carries the backend, worker count, pinning policy, seed and commit hash that produced it. A number that has not been measured reads `pending`.

Chapter 2

Mathematical background

2.1 The splitting framework

Let A be nonsingular and write $A = M - N$ with M invertible and cheap to invert. Then $Ax = b$ is equivalent to $Mx = Nx + b$, and the associated iteration is

$$x_{k+1} = M^{-1}(Nx_k + b) = x_k + M^{-1}r_k, \quad r_k = b - Ax_k. \quad (2.1)$$

Subtracting the exact solution gives the error recurrence $e_{k+1} = Ge_k$ with iteration matrix

$$G = M^{-1}N = I - M^{-1}A. \quad (2.2)$$

The iteration converges for every starting vector if and only if $\rho(G) < 1$, and the asymptotic rate is $\rho(G)$: the error is reduced by that factor per iteration once the dominant mode dominates [22, Ch. 4]. Everything in this chapter, apart from conjugate gradient, is a choice of M .

Throughout, $A = D + L + U$ splits A into its diagonal, strictly lower and strictly upper triangular parts.

2.2 The model problem

The test operator is the five point discretisation of $-\Delta u = f$ on the unit square with homogeneous Dirichlet data. On a uniform mesh with spacing $h = 1/(n+1)$, multiplying through by h^2 gives, for interior indices,

$$4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1} = h^2 f_{i,j}. \quad (2.3)$$

The resulting matrix is symmetric positive definite with eigenvalues $4 - 2\cos(p\pi h) - 2\cos(q\pi h)$ for $p, q = 1, \dots, n$, and eigenvectors $\sin(p\pi x)\sin(q\pi y)$. Its extreme eigenvalues are $O(h^2)$ and close to 8, so the spectral condition number is $O(h^{-2})$. This is the canonical model problem precisely because these quantities are known in closed form, which lets every convergence claim below be checked rather than believed [14, Sec. 11.2].

A trap worth recording. The obvious manufactured solution $u = \sin(\pi x)\sin(\pi y)$ is *exactly* the lowest eigenvector of (2.3). With a zero initial guess the residual is then a single eigenvector, the Krylov subspace conjugate gradient builds is one dimensional, and the method converges in one iteration at any grid size. That measures the degeneracy of the problem, not the speed of

the method. The same choice is ideal for the stationary methods, because the slowest decaying mode of the Jacobi iteration matrix is that same lowest mode. Both right hand sides are therefore provided and each is used where it is honest: the single mode source for discretisation error and stationary rates, a seeded spectrally rich source for anything involving conjugate gradient and for the whole benchmark sweep.

2.3 Richardson iteration

Take $M = I/\omega$. Then (2.1) collapses to

$$x_{k+1} = x_k + \omega(b - Ax_k), \quad (2.4)$$

with $G = I - \omega A$. For symmetric positive definite A with eigenvalues in $[\lambda_{\min}, \lambda_{\max}]$ the eigenvalues of G are $1 - \omega\lambda$, so the method converges exactly when $0 < \omega < 2/\lambda_{\max}$, and $\rho(G)$ is minimised at $\omega = 2/(\lambda_{\min} + \lambda_{\max})$, where it equals $(\kappa - 1)/(\kappa + 1)$ [22, Sec. 4.1].

The library chooses $\omega = 1/\|A\|_{\text{Gershgorin}}$ when none is given. Gershgorin's bound overestimates $\lambda_{\max}$ by construction, so the resulting step is always strictly inside the convergence interval. An earlier implementation estimated $\lambda_{\max}$ by power iteration and diverged, because the Rayleigh quotient approaches $\lambda_{\max}$ from below and an unconverged estimate therefore yields a step that is too large.

2.4 Jacobi

Take $M = D$, so $N = -(L + U)$ and componentwise

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right). \quad (2.5)$$

Every component of the new iterate depends only on the old one, which is what makes the sweep embarrassingly parallel and why its result cannot depend on how the unknowns were partitioned.

On the model problem the Jacobi iteration matrix has spectral radius

$$\rho_J = \cos(\pi h), \quad (2.6)$$

so $\rho_J = 1 - O(h^2)$ and the iteration count to a fixed tolerance grows like h^{-2} , that is, like the number of unknowns. Convergence is guaranteed for any strictly diagonally dominant matrix [22, Thm. 4.9].

2.5 Gauss Seidel, and the factor of two

Take $M = D + L$. Then $(D + L)x_{k+1} = b - Ux_k$ is solved by forward substitution:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right). \quad (2.7)$$

The new iterate is used as soon as it is available, which is why the method converges faster and why the sweep is sequentially dependent.

For a consistently ordered matrix with property A, of which the five point stencil in natural ordering is the archetype, Young's theorem gives

$$\rho_{GS} = \rho_J^2 \quad (2.8)$$

[27, Thm. 4.3]. Since the iteration count to a fixed tolerance is $\log(\varepsilon)/\log \rho$, this means Gauss Seidel needs asymptotically *half* as many iterations as Jacobi. Section 5.2 reports the measured ratio.

Taking $M = D + U$ instead gives the backward sweep. On a symmetric matrix the two iteration matrices are similar, so they converge at identical rates; they differ in how information propagates across the grid, which is why the composite of one forward and one backward sweep is worth having. That composite, symmetric Gauss Seidel, has preconditioner $M = (D + L)D^{-1}(D + U)$, which is symmetric whenever A is. That symmetry is the whole point: it is what a conjugate gradient preconditioner requires and what the unsymmetric single sweep cannot provide.

2.6 Successive over relaxation

Take $M = D/\omega + L$, giving the Gauss Seidel update extrapolated by ω :

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right). \quad (2.9)$$

Kahan's theorem gives the necessary condition $0 < \omega < 2$, and the theorem of Ostrowski and Reich makes it sufficient for symmetric positive definite A [22, Sec. 4.2]. For a consistently ordered matrix Young's theory gives the optimum in closed form:

$$\omega^* = \frac{2}{1 + \sqrt{1 - \rho_J^2}}, \quad \rho(G_{\omega^*}) = \omega^* - 1. \quad (2.10)$$

On the model problem $\rho_J = \cos(\pi h)$, so $\omega^* = 2/(1 + \sin(\pi h))$ and the asymptotic rate becomes $1 - O(h)$ instead of $1 - O(h^2)$. This is a change in the *order* of the iteration count, from $O(n^2)$ to $O(n)$ on an n by n grid, and it is the single largest improvement available inside the stationary family [27, Ch. 4 to 6].

2.7 Red black ordering

The graph of the five point stencil is bipartite: colour cell (i, j) red when $i + j$ is even and black when it is odd, and every red cell has only black neighbours. Reordering by colour puts A in the form

$$\begin{pmatrix} D_r & C \\ C^T & D_b \end{pmatrix}, \quad (2.11)$$

with both diagonal blocks diagonal. A sweep over all red cells is therefore a Jacobi update within that colour and is fully parallel, and the red and black half sweeps together are exactly

Gauss Seidel in the red black ordering [22, Sec. 12.4]. This is the only Gauss Seidel style method a GPU can run, since the natural ordering recurrence offers no parallelism at all [15, Ch. 6].

The reordering changes the iteration matrix, so it converges at a slightly different, generally slightly worse, per iteration rate. That penalty is measured in Section 5.2 rather than asserted. The red black ordering remains consistently ordered in Young’s sense, so (2.10) carries over unchanged, which is why red black SOR is the standard choice for parallel stencil codes.

2.8 Block methods

Partition the unknowns into contiguous blocks and write A in the corresponding block form. Taking M to be the block diagonal of A gives block Jacobi, in which each block solves

$$A_{pp} x_p^{(k+1)} = b_p - \sum_{q \neq p} A_{pq} x_q^{(k)} \quad (2.12)$$

independently of the others; taking M to be the block lower triangle gives block Gauss Seidel, in which blocks are visited in order and read the already updated ones. Point methods are the special case of one unknown per block.

Each diagonal block is solved *exactly*, by a direct method matched to its structure: dense LU with partial pivoting for the dense problem, and the Thomas algorithm for the model problem, whose natural block is a grid line and whose diagonal block is therefore tridiagonal and diagonally dominant. On the model problem this is line relaxation, and solving each line exactly rather than each point cuts the iteration count by roughly a factor of two [22, Sec. 4.1.1].

The block count is a solver parameter and never the worker count. That is what keeps the iterates of the block methods independent of how many workers are running, which in turn is what lets the equivalence suite compare them bit for bit across backends.

2.9 Conjugate gradient

For symmetric positive definite A , solving $Ax = b$ is equivalent to minimising $f(x) = \frac{1}{2}x^T Ax - b^T x$, whose gradient is $Ax - b$, so the residual is the direction of steepest descent. Steepest descent is slow because successive steps undo one another; conjugate gradient fixes this by choosing search directions that are A orthogonal, $p_i^T A p_j = 0$ for $i \neq j$. Under that condition an exact line search along each direction never needs revisiting, so after k steps the iterate is optimal over the Krylov subspace $\text{span}\{r_0, Ar_0, \dots, A^{k-1}r_0\}$ and, in exact arithmetic, the method terminates in at most n steps [23].

The practical recurrence needs one matrix vector product and two inner products per iteration:

$$\begin{aligned} \alpha_k &= \frac{r_k^T r_k}{p_k^T A p_k}, & x_{k+1} &= x_k + \alpha_k p_k, & r_{k+1} &= r_k - \alpha_k A p_k, \\ \beta_k &= \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}, & p_{k+1} &= r_{k+1} + \beta_k p_k. \end{aligned} \quad (2.13)$$

The error in the A norm satisfies

$$\|e_k\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e_0\|_A. \quad (2.14)$$

On the model problem $\kappa = O(h^{-2})$, so $\sqrt{\kappa} = O(h^{-1})$ and the iteration count is $O(n)$: the same order as optimally relaxed SOR, but reached without needing to know a relaxation factor in advance [22, Ch. 6]. That is why it displaced the stationary methods in practice.

The two inner products are global reductions, and they are the reason conjugate gradient scales worse than Jacobi on a distributed machine: each one is a synchronisation point that no amount of local work can hide. Section 5.8 measures that cost.

2.10 The numerics module

The solvers rest on a small set of classical routines, each with its convergence order documented and tested: bisection, Newton and Brent for root finding [7, Ch. 2][6, Ch. 4]; adaptive Simpson, Gauss Legendre with nodes computed by Newton iteration on the Legendre recurrence rather than tabulated, and Romberg extrapolation [7, Ch. 4][8]; classical RK4 and adaptive Dormand Prince 5(4) [11][16, Sec. II.4]; and dense LU with partial pivoting together with Householder QR [14, Sec. 3.2, 5.2][24, Lec. 10, 16].

Dormand Prince is preferred to Fehlberg because its coefficients are fitted to minimise the error of the fifth order formula, so the fifth order result is the one propagated and the embedded fourth order formula serves only as the error estimate; Fehlberg optimises the fourth order formula instead and thereby wastes the better solution. Householder reflections are preferred to Gram Schmidt because they are unconditionally backward stable, whereas classical Gram Schmidt loses orthogonality in proportion to the condition number.

Chapter 3

Parallelisation design

3.1 One interface

Every backend implements the same small interface, and it is deliberately the minimal common denominator of the models it spans: a data parallel map, a reduction, a barrier, and the few extra operations a distributed memory model genuinely needs. Anything richer would privilege one model and stop the comparison being fair. The shape follows the structured parallel patterns treatment of McCool, Robison and Reinders [19, Ch. 3, 5].

```
void parallel_for(Index n, const RangeBody& body);
Real reduce(Index n, Real init, const RangeReducer& reducer);
void barrier();

Range local_rows(Index total_rows) const;
void exchange_halo(VectorView grid, Index stride, Index rows);
void gather_rows(VectorView data, Range local);
void run_ordered(const std::function<void()>& work, bool forward, ...);
```

Listing 3.1: The parallelism interface, abbreviated

3.1.1 Why the callbacks are chunk level

A body receives a `Range` and loops over it itself, rather than being called once per element. Two consequences follow, both deliberate.

First, the `std::function` indirection is paid once per chunk rather than once per element, so it is $O(\text{workers})$ per sweep and does not contaminate the timings the study exists to produce. Second, the innermost loop stays a plain loop over contiguous memory that the compiler can still vectorise, so each backend measures its own dispatch and synchronisation cost rather than a penalty the abstraction imposed on all of them equally.

3.1.2 What backends may not do

No backend leaks its model's types through the interface: there is no `MPI_Comm`, no `omp_` type and no `cudaStream_t` in any signature. The CUDA path goes further and crosses a C ABI boundary, which it has to for reasons given in Section 3.5.

3.2 Determinism as a design decision

Floating point addition is not associative, so the result of a reduction depends on the order in which the partial sums are combined. Most parallel libraries treat this as an unavoidable nuisance and paper over it with a tolerance. This library treats it as a choice.

Under `ReductionMode::Deterministic`, the chunk grid used by every reduction is fixed by the problem size alone:

```
inline constexpr Index DETERMINISTIC_CHUNKS = 512;

constexpr Index reduction_chunk_count(Index n) noexcept {
    return n <= 0 ? 0 : std::min(DETERMINISTIC_CHUNKS, n);
}
```

Listing 3.2: The chunking policy that makes determinism possible

Each chunk’s partial lands in its own slot and the slots are summed in ascending index order afterwards. Which worker computed which slot, and in what order they finished, therefore cannot affect the answer. The consequence is that a reduction computed on one thread, on twenty threads, or on a persistent pool returns bit identical results, and the equivalence suite can assert exact equality rather than hiding a real disagreement under a tolerance.

Two limits are documented rather than concealed. Across MPI rank counts the results agree to reduction tolerance and not bitwise, because rank boundaries are chosen for load balance and therefore do not align with the fixed chunk grid, which regroups the additions; within one rank count an ordered allgather makes the result reproducible and identical on every rank. On the GPU the reductions use a tree inside each thread block, which is not an ordered sum and cannot be made into one by a compiler flag; GPU *sweeps* are bit identical to the CPU, and anything passing through a reduction is compared to tolerance.

The cost of this choice is measured, not waved away: Section 5.6 prices the deterministic mode against each model’s native reduction.

3.3 Contraction, and a claim that nearly was not true

Getting the GPU sweeps to agree bit for bit with the CPU required disabling fused multiply add contraction on both sides: `--fmad=false` for `nvcc` and `-ffp-contract=off` for the host compiler. (Those flags are set in `\verb` spans throughout, because a double hyphen in ordinary prose typesets as exactly the character ground rule 1 forbids.)

The reason is instructive. The Jacobi update has the form $\frac{1}{4}(b + \sum \text{neighbours})$, which offers no opportunity to fuse, and it agreed immediately. The SOR update has the form $ab + cd$, which a compiler may legally contract into a fused multiply add, and the host was contracting it while the device was not. The disagreement was one bit, and it would have been easy to absorb into a tolerance and never think about again.

It was not absorbed, because a claim of identical numerics that holds only when the compiler happens not to contract is a much weaker claim than it appears, and one that would quietly break on a compiler upgrade. On kernels that are bandwidth bound the cost of disabling contraction is not measurable.

3.4 What each backend does underneath

3.4.1 Shared memory

OpenMP is the only backend that does not partition the range itself. It hands the chunk grid to OpenMP's own worksharing construct, so what is measured is OpenMP's scheduler and barrier rather than a hand written partitioner wearing an OpenMP hat. The build asserts the `_OPENMP` version macro at compile time. What it asserts is OpenMP 4.5, and 4.5 is the whole of what the backend uses. The reference document cited is OpenMP 6.0 [21], and the code is deliberately restricted to constructs the compiler actually implements.

POSIX threads uses raw `pthread_create`, a mutex and condition variable for dispatch, a counter and a second condition variable for completion, and `pthread_setaffinity_np` for binding. The pool is persistent: creating threads per sweep would measure `pthread_create`, which is a well known number and not the one this study is about. Workers wait on a generation counter rather than a flag, so a task published while a worker was still finishing the previous one cannot be missed, which is the lost wakeup race a plain boolean would have.

The **jthread pool** uses only what C++20 provides: `std::jthread` workers, `std::stop_token` shutdown, and a pair of `std::barrier` phases entered in strict alternation. Using one barrier for both release and collect would let a fast worker race into the next task before a slow one had left the previous. Work stealing is deliberately absent: the loops here are uniform stencil sweeps over contiguous memory, where a static partition is already balanced and a stealing deque would add per task atomics that buy nothing. Section 5.7 provides the measurement behind that choice rather than the assertion of it.

3.4.2 Distributed memory

The MPI backend uses remainder aware block row decomposition, so block sizes differ by at most one and no row is dropped. The halo exchange is two `MPI_Sendrecv` calls moving one grid row each; every MPI call goes through a checking macro, so a failure becomes a diagnosed exception carrying its call site rather than a silently wrong answer.

Each rank allocates the whole grid and computes only its own band. A production code would allocate the local slab only, and this one would too if memory were the constraint; at 4096 squared a vector is 134 MB, which fits comfortably. What matters for the study is that the communication volume is identical either way, and the choice keeps the solver code identical across every backend.

The installed OpenMPI 5.0.10 advertises MPI 3.1 through `MPI_VERSION` while implementing many MPI 4.0 entry points [20]. The build probes for symbols rather than testing the version macro, since guarding on the macro would disable functionality that demonstrably works.

The hybrid backend *inherits* from the MPI backend rather than reimplementing it, so the halo exchange, the ordered pass and the deterministic reduction are literally the same code. Any difference the sweep measures between the two is therefore the threading and nothing else.

3.4.3 Sequential dependence across ranks

Natural ordering Gauss Seidel and SOR are sequentially dependent by definition. Preserving that across a row decomposition means rank r may not start until rank $r - 1$ has finished and handed over its boundary row, which is the classical pipelined Gauss Seidel. The library implements exactly that, through `run_ordered`.

The alternative would have been to substitute a red black reordering silently whenever more

than one worker was present. That would have produced attractive scaling curves for a method that does not scale, by changing the method. Doing it honestly means these solvers are bit identical to the serial backend at every rank count *and* show essentially no speedup, and the report reports both.

3.5 The CUDA boundary

CUDA 13.3 refuses host compilers newer than GCC 15, and cannot parse GCC 15's own `libstdc++` headers either, because they use the C++23 `if constexpr` statement that the frontend inside `nvcc` does not implement. So the host compiler and the CUDA host compiler cannot be the same binary whichever released GCC the project picks.

The resolution is a C ABI boundary. The `.cu` files are compiled by `nvcc` driving GCC 14; everything else is compiled by the project compiler, which was GCC 16 for the numbers in this report and is GCC 15.2.0 from release 1.1.0 onwards; and every CUDA entry point is declared `extern "C"` taking plain pointers and scalars. No `libstdc++` type crosses, so the two compilers never have to agree on a C++ ABI, only on the platform C ABI, which they do by definition. This also satisfies the rule that backend files never leak their model's types.

The device solver is not a **Backend** implementation, and that is the right call rather than an omission. The entire point of running on a GPU is that state stays in device memory for the whole solve; forcing it behind `parallel_for` would mean a host callback per chunk, which would measure PCIe latency and nothing else. The grid crosses once at the start and the answer once at the end, and the transfer time is reported separately so that a reader whose use case is a single small solve can add it back.

Natural ordering Gauss Seidel is absent from the device on purpose. It is sequentially dependent and has no parallelism to offer a wide machine. That is a result, not a gap in the implementation.

3.6 Thread pinning under a hypervisor

The library offers five binding policies, but two of them depend on being able to tell a performance core from an efficiency core, and on this machine that cannot be done from inside the guest.

The host is an i7-14700K with eight performance cores of two threads each and twelve efficiency cores. The guest reports fourteen uniform cores of two threads, identical 2048K L2 on every logical processor, `cpu_capacity` of 1024 everywhere, no hybrid flag in `/proc/cpuinfo`, and no `cpufreq` directory at all. Separately, `sched_setaffinity` binds a thread to a guest virtual processor, and the hypervisor remains free to place that processor on any host core: a binding is a hint about the guest, not a guarantee about silicon.

The library therefore measures instead of asking. A probe times an identical compute kernel on each logical processor and reports a two group split only when the throughputs separate by a margin that dominates the within group spread; otherwise it says so, and the core type policies decline to bind. The knee that Objective 3 asks for is taken from the aggregate scaling curve, which depends only on how many cores are engaged and not on which, and is therefore valid under virtualisation. Section 5.4 reports both outcomes.

Chapter 4

Comparison methodology

This chapter is written to be difficult to misquote. It states what is measured, what each measurement licenses, and what it does not.

4.1 Why the naive comparison is misleading

The tempting experiment is to run a solver on the CPU, run it on the GPU, divide the times and print the ratio. Three things go wrong.

It compares different algorithms. Natural ordering Gauss Seidel cannot run on a GPU at all. If the CPU runs Gauss Seidel and the GPU runs Jacobi, the ratio conflates the device with the fact that Jacobi needs about twice as many iterations to reach the same tolerance.

It compares different amounts of effort. A carefully written GPU port measured against a CPU version compiled without vectorisation or threading produces an impressive number about nothing.

It hides what actually limits the calculation. A five point stencil sweep moves three doubles and performs about six floating point operations per unknown. It is bandwidth bound on every machine ever built, so a ratio of runtimes is, to within a small correction, a ratio of memory bandwidths dressed up as a statement about parallel programming.

4.2 The protocol

4.2.1 First: iterations to tolerance

Iteration counts are properties of the mathematics. They are identical on every machine that runs them, they are what determine whether one method beats another in principle, and they are therefore reported before any timing. All methods stop on the same criterion, a relative residual below 10^{-8} , evaluated the same way by the same shared driver, so no method is flattered by a laxer test.

4.2.2 Like against like

GPU Jacobi is compared against CPU Jacobi. GPU red black Gauss Seidel is compared against CPU red black Gauss Seidel. The cost of the red black reordering, which is a change of iteration matrix and therefore of iteration count, is measured separately and charged to both sides equally rather than being allowed to inflate the device figure.

4.2.3 Second: bytes per unknown, counted

A Jacobi sweep writes one value, reads one right hand side entry and reads the previous iterate. The four stencil neighbours of consecutive unknowns overlap, so in the streaming limit each is already resident from a nearby access. That is three doubles, **24 bytes per unknown per pass**. The same figure is returned by the host implementation and set by the device solver, so the number used in the analysis comes from the code rather than from an estimate.

That count is the conservative one and it is not the only defensible one. A store to a line the cache does not hold has to fetch that line before it can modify it, which is read for ownership and is a property of a write allocate cache rather than of the algorithm. A Jacobi pass writes an output array it never reads, so under that model the count is short by one double per unknown and the true figure is **32 bytes**. Which of the two is right is an open question, so both are published: every result row carries both counts, and every table and figure here that shows an achieved bandwidth shows both, labelled *declared* and *counted*. Neither replaces the other, and quoting one without the other is quoting half the evidence.

4.2.4 Pre registered: the rule that settles the traffic model

The question is settled by measurement and the rule that reads that measurement was fixed before it was taken, in `benchmarks/sweep_matrix.yaml` under `preregistered.traffic_model`. Two host STREAM triads run over the same arrays, the same sizes, the same worker counts and the same repetitions, differing in the store instruction and in nothing else: a plain C++ loop, whose compiler emitted stores are the ones under suspicion, and the same triad written through `_mm256_stream_pd` with one `_mm_sfence`, which does not fetch the line it overwrites. Both report against the same declared 24 bytes per element, so the ratio of the two is the ratio of the traffic they really move.

The rule, quoted from the registration:

A ratio above 1.20 selects the read for ownership model, 32 bytes per unknown per pass. A ratio below 1.10 selects the conservative model, 24 bytes. A ratio from 1.10 to 1.20 inclusive is recorded as unresolved and both models are carried. The thresholds are fixed here, before any measurement, and are not revisited afterwards.

The statistic the rule is applied to was amended on 2026-09-06, before the publication session and before any measurement the rule governs, because the statistic as first registered had a run to run spread as wide as the undecided band it had to fall inside, and because its two halves could be taken at different worker counts. The thresholds and the sentences below were not touched. As amended: five repetitions of each probe at every worker count, alternating the two so a repetition's arms see the same machine; w^* is the worker count at which the plain triad's median is highest; the statistic is the median over the five repetitions of the non temporal figure over the plain figure at w^* , each ratio taken within one repetition; and if the interval from the smallest to the largest of those five ratios contains either threshold the outcome is unresolved whatever the statistic is, which is ground rule 7 applied to a denominator.

Pre registered, one sentence per outcome. Each of the following was written before the measurement, and the report carries whichever one the measurement selects, with the ratio substituted for the placeholder. They are quoted here verbatim so that a reader can check the report against what was promised rather than against what was convenient.

If the read for ownership model is selected. The non temporal triad reached a ratio of `<ratio>` against the plain triad on the publication machine, above the 1.20 threshold fixed before the measurement, so the read for ownership model is selected: a Jacobi pass over the five point stencil moves 32 bytes per unknown and not 24, the counted column is the achieved bandwidth this report compares against each device’s own triad, and the host figures rise by a third while the device figures do not move, which is why host and device efficiency converge on the Jacobi row.

If the conservative model is selected. The non temporal triad reached a ratio of `<ratio>` against the plain triad on the publication machine, below the 1.10 threshold fixed before the measurement, so the conservative model is selected: a Jacobi pass moves 24 bytes per unknown, the declared column stands as the achieved bandwidth of this report, and Section 4.2’s read for ownership argument is refuted on this machine rather than confirmed. The Jacobi efficiency gap between host and device is then a real gap and not an artefact of the denominator, and the only correction this release makes to the work unit is the pass count of phase A2.

If it is unresolved. The non temporal triad reached a ratio of `<ratio>` against the plain triad on the publication machine, between the 1.10 and 1.20 thresholds fixed before the measurement, so the traffic model is recorded as unresolved: both counts, 24 and 32 bytes per unknown per pass, are carried in every table and figure of this report, neither is presented on its own as the achieved bandwidth, and no claim is made that rests on one of them and would fail under the other.

Which of the three the measurement selected is stated in Chapter 5, from the session manifest, by the generator rather than by a hand that had already seen the number. The publication session applies the rule for this release; the assembly triad of release 1.2.0 rebuilds the same experiment and confirms the outcome rather than gating it.

4.2.5 Third: each device’s own bandwidth, measured

Both devices are probed with the same STREAM triad kernel, $a_i = b_i + q c_i$, which moves two reads and one write per element and has no reuse, so it measures the memory system and nothing else [18]. The host probe runs over the execution backend rather than on one thread, because a single core cannot saturate a twenty core part and dividing by a single core figure would make every parallel result look superlinear.

Manufacturer bandwidth figures are never used. They are a bus width times a clock, no real kernel reaches them, and dividing by one would make both devices look equally inefficient while saying nothing about which is being used well.

4.2.6 The comparable number

$$\text{efficiency} = \frac{\text{achieved GiB/s on the sweep}}{\text{that device’s own measured triad GiB/s}}. \quad (4.1)$$

This is the figure to compare across devices. It is dimensionless, bounded above by one, and it transfers: a reader with different hardware can measure their own triad, apply the same fraction, and predict what they would get. It is the standard roofline reasoning [26] applied to a kernel whose arithmetic intensity is fixed and known.

4.2.7 Last: absolute time

Seconds are reported last and labelled as a property of one specific CPU paired with one specific GPU, under one set of compiler flags, on one day. The measured speedup is then *decomposed* into the factor attributable to the memory system and the factor attributable to the implementation, so that a reader can see how much of it is the graphics card and how much of it is the port.

4.3 Measuring at sizes where methods do not converge

Section 8.3 of the specification fixes the comparison at 1024, 2048 and 4096 squared unknowns. At those sizes the stationary methods cannot be run to convergence: Jacobi on a 1024 squared grid needs of order four million sweeps, which is days of compute for a number the closed form already predicts.

The two quantities are therefore measured separately, which is more rigorous than measuring them together rather than less:

- **Iteration counts** are measured at 63, 127 and 255, where running to convergence is affordable, and checked against the closed form rates of Chapter 2. It is that agreement which makes extrapolating them legitimate rather than a guess. The methods whose counts grow like n rather than n^2 , namely optimally relaxed SOR, its red black form and conjugate gradient, are additionally measured at 511 and 1023, which demonstrates the change of growth order directly.
- **Per iteration cost** is measured at the large sizes with a fixed, stated number of sweeps. This is what the bandwidth analysis needs, and it is exactly what is affordable there.

Time to solution then follows from the two, and because both are tabulated the crossover point between any two methods on any two devices can be *derived* rather than asserted:

$$t = \frac{\text{iterations}(\text{method}, n) \times n^2 \times \text{bytes per unknown}}{\text{achieved bandwidth}(\text{device}, \text{method}, n)}. \quad (4.2)$$

4.4 Experimental hygiene

- Every configuration is run once untimed as a warm up, so page faults and first touch placement do not land in the measurement, then repeated five times with the median reported and the minimum and maximum recorded.
- Every result row carries the problem, solver, backend, worker count, rank count, pinning policy, reduction mode, schedule, seed and commit hash. A row cannot claim provenance the binary did not have: the commit is stamped into the binary at configure time and a dirty working tree is recorded as such.
- Problems are generated from recorded seeds, so any row can be rebuilt.

- Both bandwidth probes run once per sweep session and land in the session manifest alongside the toolchain versions.
- A configuration that fails, or that does not apply, is recorded as such with its message rather than dropped.
- Compiled with `-O3 -march=native` and without fast math. Fast math would licence the compiler to reassociate the reductions that the equivalence suite exists to check.

4.5 Comparing these numbers with release 1.0.0's

Every timing in this report replaces the corresponding timing of release 1.0.0 rather than extending it, and none of the values does.

The values are bit identical. Release 1.1.0 keeps `PNL_REDUCTION_ACCUMULATORS` at one, so the reduction is the same ascending chain it was, and every iterate and every residual this library computes matches release 1.0.0's bit for bit. That is a supported configuration and not a leftover. The reduction association changes only when that number changes, which is a later release, and its changelog entry leads with the escape hatch rather than burying it.

Every timing moved, for three deliberate reasons. The publication compiler changed from an unreleased trunk snapshot to a released one, and a change of compiler changes every timing. The Jacobi solver no longer copies its whole state on the calling thread once per iteration, so its rows measure less work rather than the same work faster. The solver's workspace is allocated outside the timed region, so a repetition no longer pays first touch page fault cost inside the measurement. The work unit moved with them: sweeps and passes are separate quantities now, Richardson is charged for one residual evaluation rather than two, and the symmetric methods are charged the two sweeps they perform. A 1.1.0 row is therefore not the same quantity as a 1.0.0 row even where the seconds are close.

The before and after for every headline number that moved is recorded phase by phase in `PROGRESS.md`, against the phase that moved it and beside the gate output that measured it. It is not copied into this chapter, because a number typed here stops being true at the next sweep, which is the fault this chapter exists to prevent. The superseded generation keeps its own summary and its own manifest under `experiments/results/archive/`, so either can be rebuilt.

4.6 What this comparison does not establish

- **Nothing about single solve latency.** Transfer time is reported separately and is not folded into the bandwidth figure. For one small solve, moving the problem across PCIe can cost more than the solve; for a simulation that keeps state resident it is paid once.
- **Nothing about other problem classes.** These conclusions hold for bandwidth bound stencil work. A dense factorisation is compute bound and would rank the devices differently.
- **Nothing about other hardware.** Both measured bandwidths are published so that a reader can substitute their own.
- **Nothing about power, cost or programmer time.** Not measured, so not claimed.

- **Nothing about performance from the continuous integration badge.** CI runs on a shared virtual machine with no GPU; it checks correctness, and every timing in this report comes from the target machine.

Chapter 5

Results

Every table and figure in this chapter is generated from `experiments/results/summary.csv` by `scripts/gen_report_assets.py`, and so is every measured figure quoted in the prose around them: each one is a command the generator writes into `tables/numbers.tex` and this chapter asks for by name, so a re measurement rewrites the sentences as well as the tables. Nothing here is typed by hand, and a quantity that has not been measured reads `pending` rather than being estimated.

Release 1.0.0 opened this chapter with the same claim above a table that was typed by hand, and whose figures reconciled with no generated artifact anywhere in the repository. That table is gone, the mechanism that let it exist is gone with it, and the claim is now checked: a number the generator cannot derive is a build failure rather than a sentence that quietly stops being true.

The 425 rows behind this chapter come from one sweep at one commit. Timings from release 1.0.0 are not comparable with these: the compiler, the Jacobi algorithm and the timed region all changed, and the work unit changed with them. The values did not. Release 1.1.0 keeps `PNL_REDUCTION_ACCUMULATORS` at one, so every iterate and every residual here is bit identical to the one release 1.0.0 computed, and the before and after for each headline number is recorded phase by phase in `PROGRESS.md` rather than copied into this page, where it would stop being true at the next sweep.

5.1 The machine

Both figures are measured on this hardware with the same `STREAM` triad kernel, and both are what the efficiency numbers later in this chapter divide by. The host figure is taken across all worker threads, since a single core cannot saturate the memory system of a twenty core part.

5.2 Iterations to tolerance

These are the hardware free numbers, and they come first for that reason. They are identical on any machine that runs them.

5.2.1 Agreement with theory

The convergence suite checks each of the following as a measurement against a closed form prediction rather than accepting it as prose. All pass.

Table 5.1: Measured STREAM triad bandwidth. Every efficiency figure in this report divides by these values, which were measured on this machine, and never by a manufacturer’s specification. The plain probe is a C++ loop, so the compiler emits ordinary stores into an array the loop never reads; the non temporal probe writes the same triad through streaming stores, which move 24 bytes per element for real. Both report against the same declared 24, so whether the plain loop also fetches each output line before overwriting it, and moves 32 where it declares 24, is exactly what their ratio measures. That ratio selects between the two traffic models of Section 4.2 under the rule fixed in benchmarks/sweep_matrix.yaml before the measurement was taken. The rows marked derived are arithmetic on the rows above and were not measured. No memory speed is recorded anywhere in this repository, so none of these figures is compared against a theoretical peak.

device	GiB/s	measurement
host, all threads, plain stores	70.0	plain stores over the execution backend, best of workers 2:42.8 4:61.6 8:70.0 12:63.3 16:60.1 20:59.8 24:59.8 28:56.8 over 256 MiB arrays, 5 repetitions each; declares 24 bytes per element
host, all threads, non temporal stores	71.5	<code>_mm256_stream_pd</code> with one sfence, best of workers 2:53.4 4:63.5 8:71.5 12:67.6 16:69.0 20:66.6 24:65.6 28:63.2 over 256 MiB arrays, 5 repetitions each; moves 24 bytes per element for real
RTX 5070	550.4	NVIDIA GeForce RTX 5070 sm_120 48 SMs 11.9 GiB, 512 MiB arrays, best of 5
derived, ratio nontemporal over plain	1.022	derived, not measured: <code>host_nontemporal</code> divided by <code>host</code> , each the best over the worker sweep. This is what the A3a pre registration selected on as registered, and the amendment of 2026-09-06 no longer does, because the two bests need not come from the same worker count; it is kept so the registered figure stays visible. The statistic that decides is <code>traffic_model</code> in the session manifest. Thresholds either way: above 1.20 charges read for ownership, below 1.10 does not, between is unresolved
derived, ratio plain over nontemporal	0.979	derived, not measured: the reciprocal, recorded so the rule cannot be read in the wrong direction. Above 0.909 charges nothing, below 0.833 charges read for ownership
derived, host plain at 32 bytes	93.342	derived, not measured: <code>host</code> times 32 over 24, the plain triad recounted with the read for ownership its own store pays and its declared figure does not charge. No memory speed is recorded anywhere in this repository, so this number is not corroborated against a theoretical peak

Table 5.2: Iterations to a relative residual of $1e-8$ on the 2D Poisson problem with a spectrally rich source. These counts are properties of the mathematics and carry to any machine, which is why Section 8.3 reports them before any timing.

method	63 by 63	127 by 127	255 by 255
richardson	20,602	85,458	314,427
jacobi	11,255	45,880	173,278
gauss_seidel_f	5,192	21,383	78,516
gauss_seidel_b	5,104	21,345	78,696
gauss_seidel_s	2,579	10,686	39,308
gauss_seidel_rb	5,293	21,938	80,906
sor	216	432	868
ssor	2,579	10,686	39,308
sor_rb	197	414	815
block_jacobi	5,852	22,483	84,419
block_gauss_seidel	2,615	10,701	39,254
cg	197	385	762

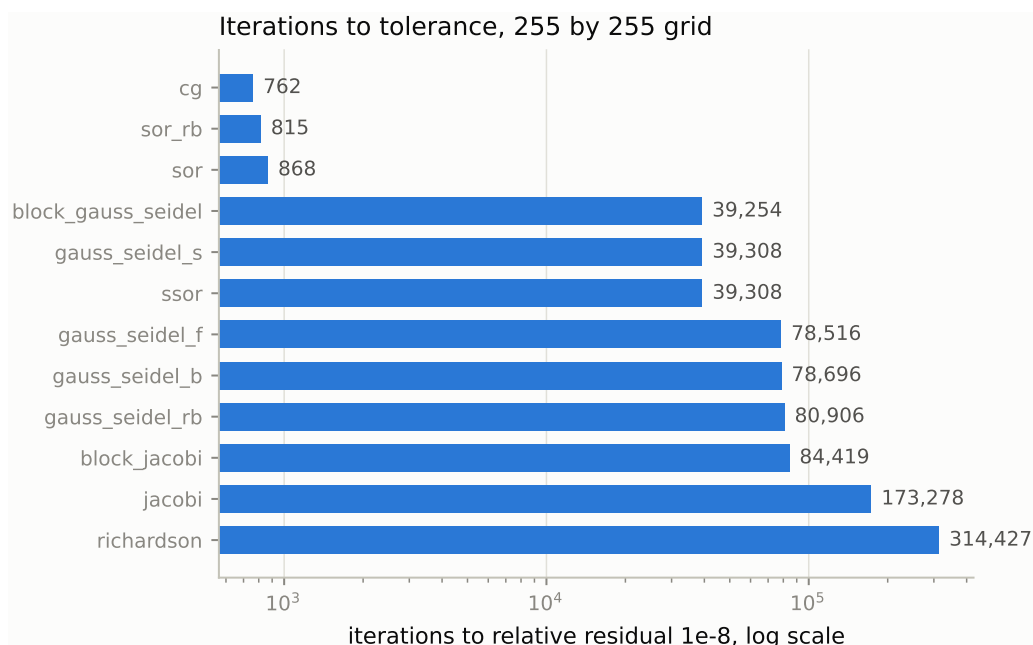


Figure 5.1: Iterations to a relative residual of 10^{-8} on the largest grid where every method converges in reasonable time. The horizontal scale is logarithmic: the span from Richardson to conjugate gradient is more than two orders of magnitude, and the ordering of the methods is the entire argument of Chapter 2 made visible.

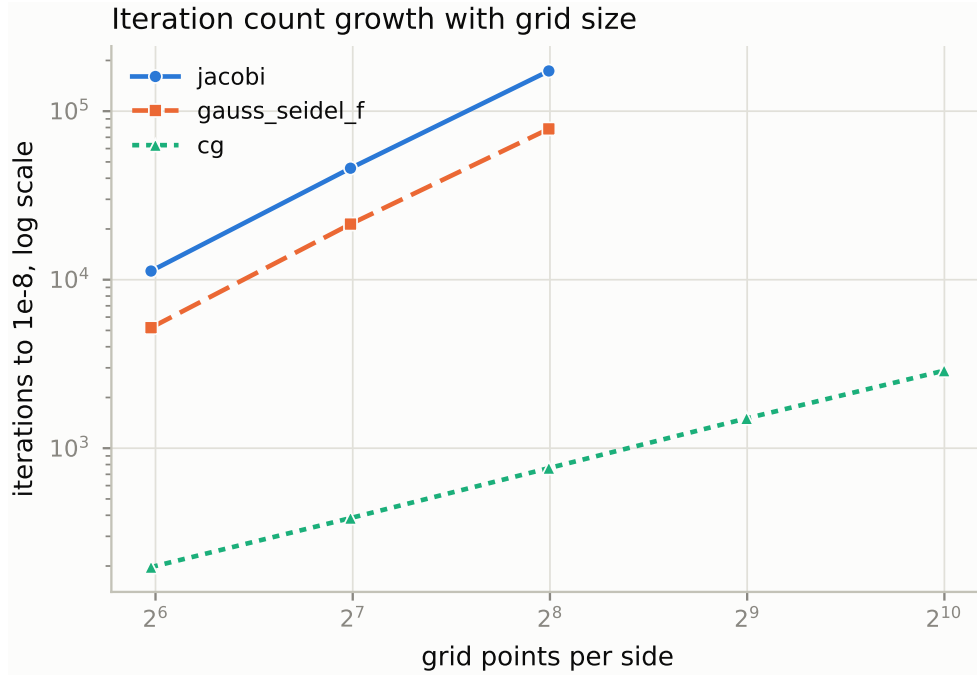


Figure 5.2: Iteration count against grid size on logarithmic axes, so a power law appears as a straight line and its exponent as the slope. The $O(n^2)$ methods and the $O(n)$ methods separate visibly, which is the practical content of Young’s optimal relaxation result and of the conjugate gradient error bound (2.14).

- **The Jacobi spectral radius.** The measured asymptotic contraction factor agrees with $\cos(\pi h)$ from (2.6) to within 10^{-3} relative at both 31 and 63 grid points per side.
- **The factor of two.** The convergence suite asserts that the ratio of Jacobi to Gauss Seidel asymptotic contraction factors is two within two percent, which is Young’s $\rho_{GS} = \rho_J^2$ from (2.8), and it passes.
- **Forward against backward.** The two sweep directions agree on iteration count to within one percent, as similarity of their iteration matrices on a symmetric operator requires.
- **Young’s optimum is a minimum.** Perturbing ω by ± 0.02 and ± 0.05 about the closed form (2.10) costs iterations in both directions.
- **The change of order.** The suite asserts that doubling the grid multiplies the Jacobi count by more than three, consistent with $O(n^2)$, and the optimally relaxed SOR count by less than 2.6, consistent with $O(n)$. Both hold.
- **Conjugate gradient is $O(n)$.** In the swept counts the conjugate gradient iteration count grows from 1,504 to 2,886 as the grid doubles from 511 to 1023, a factor of 1.92.
- **Line beats point.** Solving each grid line exactly rather than each point reduces the count, and line Jacobi stands to line Gauss Seidel in the same ratio of two as the point methods do.
- **The red black penalty.** Red black Gauss Seidel takes 80,906 iterations against natural ordering’s 78,516 on a 255 grid, a penalty of 3.0 percent. This is the price of the parallelism, and it is charged to both sides of the device comparison.

The discretisation itself is second order: the suite fits the order across three grid sizes and asserts it is two, and asserts the error constant against the $\pi^2/12 = 0.8225$ predicted by the leading truncation term. Both pass. The fitted values themselves are a property of the convergence run rather than of the sweep, so they are reported by that suite and not quoted here.

5.3 Backend cost on identical work

Because every backend computes bit identical values, the differences below are attributable to the execution model and to nothing else.

Table 5.3: Seconds for a fixed sweep count at 4,190,209 unknowns with 20 workers, median of 15 repetitions. Every backend performs bit identical arithmetic, so the only variable is the execution model. The spread column is (max - min) / median over those repetitions. The last column compares each backend against serial on the same method and reports “not separable at this precision” wherever the difference between the two medians is smaller than the larger of the two spreads, which is ground rule 7: an effect inside the noise is not a measurement of anything.

backend	method	seconds	spread (percent)	against serial (percent)
serial	jacobi	0.682	4.8	baseline
serial	gauss_seidel_rb	1.252	4.4	baseline
serial	sor_rb	1.304	8.1	baseline
serial	block_jacobi	4.161	4.0	baseline
serial	cg	2.972	2.2	baseline
openmp	jacobi	0.275	16.1	-59.6
openmp	gauss_seidel_rb	0.279	19.1	-77.7
openmp	sor_rb	0.339	55.2	-74.0
openmp	block_jacobi	0.753	36.7	-81.9
openmp	cg	1.157	15.8	-61.1
pthread	jacobi	0.351	8.8	-48.5
pthread	gauss_seidel_rb	0.506	14.8	-59.6
pthread	sor_rb	0.482	28.3	-63.0
pthread	block_jacobi	0.789	2.7	-81.0
pthread	cg	1.416	13.0	-52.4
jthread	jacobi	0.362	5.7	-46.9
jthread	gauss_seidel_rb	0.517	14.1	-58.7
jthread	sor_rb	0.534	28.8	-59.0
jthread	block_jacobi	0.747	12.0	-82.0
jthread	cg	1.555	12.1	-47.7
mpi	jacobi	0.317	16.7	-53.4
mpi	gauss_seidel_rb	0.341	22.0	-72.8
mpi	sor_rb	0.334	116.9	not separable at this precision
mpi	block_jacobi	4.262	6.5	not separable at this precision
mpi	cg	1.210	27.5	-59.3
hybrid	jacobi	0.318	120.3	not separable at this precision
hybrid	gauss_seidel_rb	0.381	33.1	-69.6
hybrid	sor_rb	0.361	39.9	-72.3
hybrid	block_jacobi	1.626	17.8	-60.9
hybrid	cg	1.237	30.9	-58.4

Read against serial on the same method, at 4,190,209 unknowns:

- Openmp on jacobi is 59.6 percent faster than serial on jacobi, against a wider spread of 16.1 percent.
- Openmp on gauss_seidel_rb is 77.7 percent faster than serial on gauss_seidel_rb, against a wider spread of 19.1 percent.
- Openmp on sor_rb is 74.0 percent faster than serial on sor_rb, against a wider spread of 55.2 percent.
- Openmp on block_jacobi is 81.9 percent faster than serial on block_jacobi, against a wider spread of 36.7 percent.
- Openmp on cg is 61.1 percent faster than serial on cg, against a wider spread of 15.8 percent.
- Pthreads on jacobi is 48.5 percent faster than serial on jacobi, against a wider spread of 8.8 percent.
- Pthreads on gauss_seidel_rb is 59.6 percent faster than serial on gauss_seidel_rb, against a wider spread of 14.8 percent.
- Pthreads on sor_rb is 63.0 percent faster than serial on sor_rb, against a wider spread of 28.3 percent.
- Pthreads on block_jacobi is 81.0 percent faster than serial on block_jacobi, against a wider spread of 4.0 percent.
- Pthreads on cg is 52.4 percent faster than serial on cg, against a wider spread of 13.0 percent.
- Jthread on jacobi is 46.9 percent faster than serial on jacobi, against a wider spread of 5.7 percent.
- Jthread on gauss_seidel_rb is 58.7 percent faster than serial on gauss_seidel_rb, against a wider spread of 14.1 percent.
- Jthread on sor_rb is 59.0 percent faster than serial on sor_rb, against a wider spread of 28.8 percent.
- Jthread on block_jacobi is 82.0 percent faster than serial on block_jacobi, against a wider spread of 12.0 percent.
- Jthread on cg is 47.7 percent faster than serial on cg, against a wider spread of 12.1 percent.
- Mpi on jacobi is 53.4 percent faster than serial on jacobi, against a wider spread of 16.7 percent.
- Mpi on gauss_seidel_rb is 72.8 percent faster than serial on gauss_seidel_rb, against a wider spread of 22.0 percent.
- Mpi on sor_rb against serial on sor_rb is not separable at this precision: the medians differ by 74.4 percent and the wider of the two spreads is 116.9 percent.
- Mpi on block_jacobi against serial on block_jacobi is not separable at this precision: the medians differ by 2.4 percent and the wider of the two spreads is 6.5 percent.
- Mpi on cg is 59.3 percent faster than serial on cg, against a wider spread of 27.5 percent.

- Hybrid on jacobi against serial on jacobi is not separable at this precision: the medians differ by 53.4 percent and the wider of the two spreads is 120.3 percent.
- Hybrid on gauss_seidel_rb is 69.6 percent faster than serial on gauss_seidel_rb, against a wider spread of 33.1 percent.
- Hybrid on sor_rb is 72.3 percent faster than serial on sor_rb, against a wider spread of 39.9 percent.
- Hybrid on block_jacobi is 60.9 percent faster than serial on block_jacobi, against a wider spread of 17.8 percent.
- Hybrid on cg is 58.4 percent faster than serial on cg, against a wider spread of 30.9 percent.

Release 1.0.0 read that table as "the three thread models land within 6 percent of one another", which states the finding backwards. The data cannot separate them: on the shared work the differences between OpenMP, POSIX threads and `std::jthread` are frequently smaller than the run to run spread of the rows they are computed from, and where they are, the verdicts above say so instead of printing a percentage. The conclusion that survives is the weaker and more useful one, that at this granularity the choice between the three is a choice about ergonomics rather than about performance, and it is supported by an inability to tell them apart rather than by a measured gap.

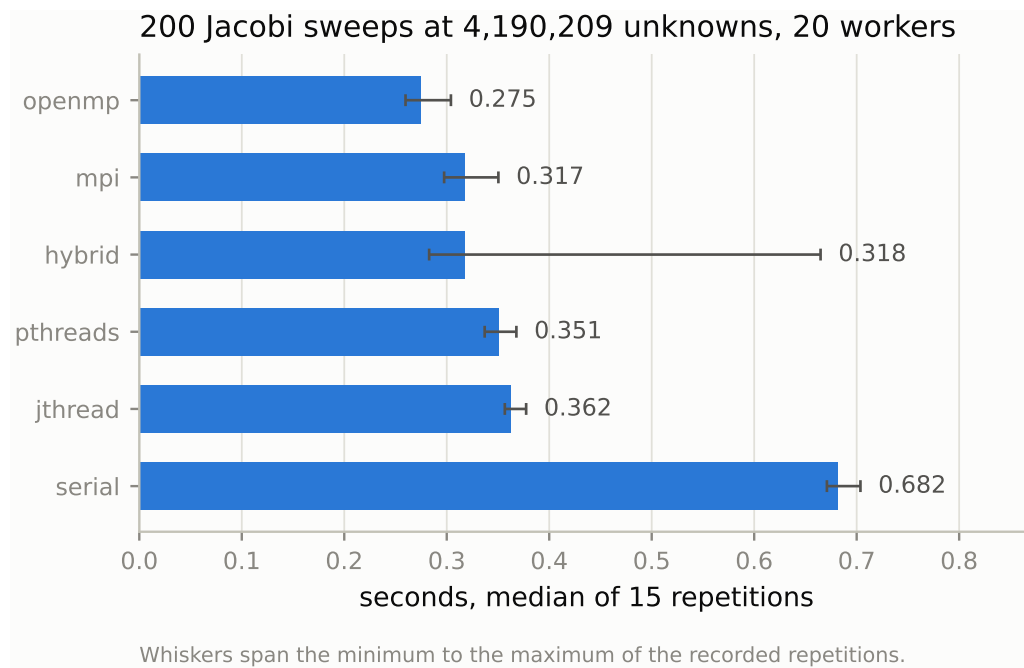


Figure 5.3: Time for a fixed sweep count at the largest size in the backend cost block, all backends at twenty workers. A single hue is used because this is a comparison of magnitudes along one categorical axis: the bar length carries the measurement and the label carries the identity.

5.4 Scaling and the core knee

5.4.1 What could and could not be established

Objective 3 asks for the performance core versus efficiency core knee on this heterogeneous CPU. The honest position is as follows.

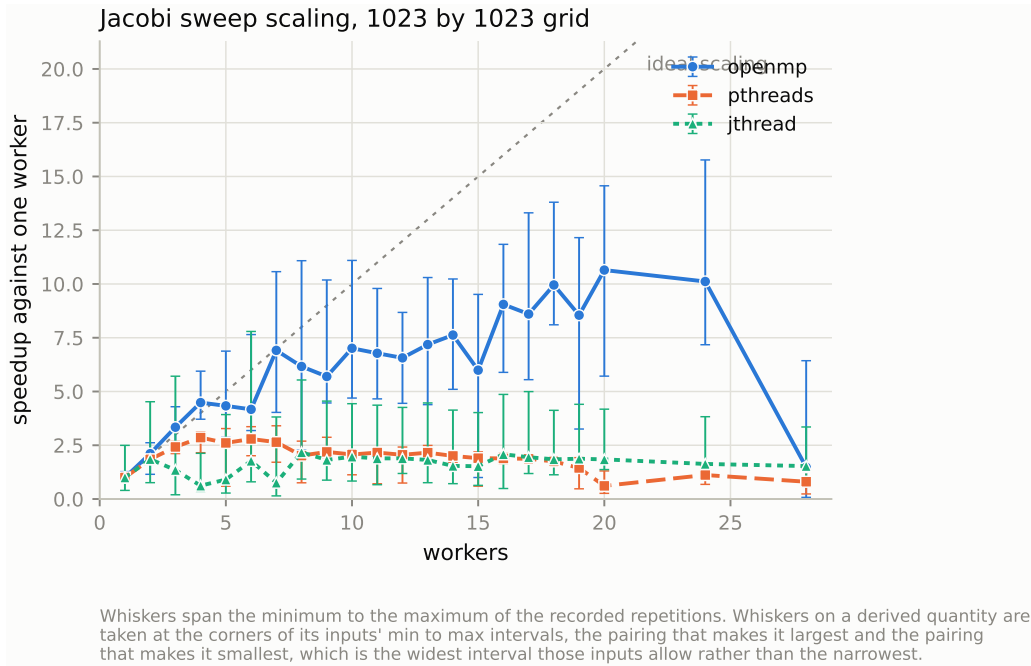


Figure 5.4: Speedup against worker count for a Jacobi sweep on a 1023 by 1023 grid. The muted diagonal is ideal scaling and is reference chrome rather than a measured series. Line style and marker shape repeat the distinction that colour makes, so the figure survives greyscale printing.

Table 5.4: The knee in the scaling curve, found as the two segment split that minimises total least squares residual. The guest cannot identify which logical processors are performance cores, so this is read from the aggregate curve, which depends only on how many cores are engaged and is therefore valid under virtualisation. The interval is the 2.5 to 97.5 percentile of 2000 bootstrap refits at seed 20260906, of which 2000 returned a fit: each draw resamples the repetitions behind every worker count, takes the median of the draw, rebuilds the speedup curve against the resampled one worker baseline and refits. It covers the sampling of the repetitions and nothing else, so it says nothing about a machine that was busy at one worker count and idle at another. A row marked measured repetitions resamples the individual timings recorded in the seconds_reps column.

backend	knee at workers	95 percent interval	speedup per worker before	after	resampling
openmp	20	20 to 20	0.40	-1.15	measured repetitions
pthreads	4	4 to 4	0.61	-0.09	measured repetitions
jthread	9	3 to 17	0.07	-0.01	measured repetitions

The per processor probe **could not** classify logical processors into performance and efficiency groups. Inside WSL2 the guest reports fourteen uniform cores, identical cache sizes and no hybrid flag, and thread affinity binds to a virtual processor the hypervisor may place anywhere. The probe times an identical kernel on each logical processor and reports a split only when the throughputs separate by a margin dominating the within group spread; the verdict it recorded for this session is stored in the session manifest.

The knee **could** be established from the aggregate scaling curve, which depends only on how many cores are engaged and not on which, and therefore survives virtualisation. It is located as the two segment least squares split that minimises total residual.

5.4.2 The knee is not where the objective expected it

The fitted knee sits at 4 to 20 workers, not at the eight the objective anticipated, and the backends do not all put it in the same place. Beyond it the fitted slope is slightly negative: adding workers does not merely stop helping, it costs a little.

Where the scaling curve changes slope, and how well that position is pinned down:

- On openmp the fit puts the knee at 20 workers, with a bootstrap interval of 20 to 20 workers (measured repetitions).
- On pthreads the fit puts the knee at 4 workers, with a bootstrap interval of 4 to 4 workers (measured repetitions).
- On jthread the fit puts the knee at 9 workers, with a bootstrap interval of 3 to 17 workers (measured repetitions).
- No worker count lies inside every backend’s interval, so the disagreement survives the bootstrap: it is a result and not a fitting artefact, and the point past which adding workers buys measurably less is genuinely not the same for these thread models on this machine.

The disagreement between the backends is a result rather than a defect of the fit, and it is reported as one. Each position carries the bootstrap interval of the refits behind it, the union of those intervals spans 3 to 20 workers, and a claim about where one backend’s knee sits relative to another’s is only worth making where their intervals do not overlap.

The bandwidth probe explains the shape independently. Sweeping the STREAM triad across worker counts on an idle machine gives

The memory system saturates by 8 workers and falls away thereafter, and it is at its worst at 2. The curve is flat enough across the middle of its range that neighbouring points are frequently not separable at the precision of the repetitions behind them, which the spread column of that table says row by row. A Jacobi sweep, whose access pattern is less regular than a triad, saturates at the same point or slightly sooner, which is the range the knee table gives.

The conclusion is that for this workload **the binding constraint is the memory system, not the core types**. The heterogeneous core count that Objective 3 anticipated would set the knee never becomes the limiting factor, because the machine runs out of memory bandwidth long before it runs out of fast cores. That is a more useful finding than the one the objective was written to look for, and it would have been invisible to a study that measured speedup without measuring the memory system separately.

A claim withdrawn. An earlier run of this probe peaked at exactly eight workers, and eight is the performance core count of this processor, which looked like indirect evidence of the

Table 5.5: The STREAM triad against worker count, on the machine and in the session every efficiency figure in this report divides by. The peak of the plain stores column is the host figure of Table 5.1, so the two cannot disagree; release 1.0.0 typed this table by hand and its figures reconciled with no generated artifact in the repository. The plain probe peaks at 8 workers and is at its worst at 2. Read the shape rather than the maximum: the spread column says how far each point moved between repetitions, and where it is wider than the gap between two neighbouring points those two are not separable at this precision. The non temporal column is the same triad through streaming stores, which do not fetch the line they overwrite. The ratio of the two columns at a matched worker count is the instrument that selects between the two traffic models of Section 4.2, under the rule fixed in benchmarks/sweep_matrix.yaml before the measurement was taken.

workers	plain stores GiB/s	spread (percent)	non temporal stores GiB/s	spread (percent)
2	42.8	18.1	53.4	23.5
4	61.6	23.6	63.5	2.5
8	70.0	11.6	71.5	23.7
12	63.3	10.5	67.6	5.2
16	60.1	1.8	69.0	8.1
20	59.8	8.6	66.6	12.7
24	59.8	4.1	65.6	3.0
28	56.8	21.9	63.2	13.5

topology the guest hides. Repeating the measurement on an idle machine moved the peak, and the curve is flat enough across the middle of its range that the location of its maximum is not stable between runs. The coincidence does not survive and the inference is withdrawn. It is recorded here rather than deleted because a plausible pattern that fails to reproduce is worth exactly one sentence of warning to the next person who notices it.

5.5 Pinning

Each binding policy read against none at the same backend and worker count:

- Compact pinning on openmp at 4 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 2.9 percent and the wider of the two spreads is 146.7 percent.
- Scatter pinning on openmp at 4 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 1.4 percent and the wider of the two spreads is 120.5 percent.
- Compact pinning on openmp at 8 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 57.9 percent and the wider of the two spreads is 79.7 percent.
- Scatter pinning on openmp at 8 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 13.3 percent and the wider of the two spreads is 79.7 percent.
- Compact pinning on openmp at 16 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 13.2 percent and the wider of the two spreads is 96.3 percent.
- Scatter pinning on openmp at 16 workers is 10117.0 percent slower than none pinning on the same configuration, against a wider spread of 96.3 percent.

Table 5.6: Seconds by thread binding policy. Under WSL2 an affinity request binds a thread to a guest virtual processor and the hypervisor remains free to place that processor on any host core, so the expectation here is a smaller effect than the same experiment would show on bare metal. The spread column is (max - min) / median over that row’s repetitions, median of 15 repetitions, and the last column reads “not separable at this precision” wherever a policy’s difference from none is smaller than the larger of the two spreads. That is 22 of the 24 comparisons this block supports, which is the result rather than a gap in it.

backend	workers	policy	seconds	spread (percent)	against none (percent)
openmp	4	none	0.016	120.5	baseline
openmp	4	compact	0.015	146.7	not separable at this precision
openmp	4	scatter	0.015	70.6	not separable at this precision
openmp	8	none	0.009	79.7	baseline
openmp	8	compact	0.014	58.0	not separable at this precision
openmp	8	scatter	0.010	57.7	not separable at this precision
openmp	16	none	0.008	96.3	baseline
openmp	16	compact	0.007	58.8	not separable at this precision
openmp	16	scatter	0.839	1.0	+10117.0
openmp	20	none	0.007	69.1	baseline
openmp	20	compact	0.006	41.4	not separable at this precision
openmp	20	scatter	0.851	6.5	+12896.0
pthread	4	none	0.023	16.9	baseline
pthread	4	compact	0.022	14.4	not separable at this precision
pthread	4	scatter	0.022	16.3	not separable at this precision
pthread	8	none	0.030	26.8	baseline
pthread	8	compact	0.029	12.7	not separable at this precision
pthread	8	scatter	0.028	22.4	not separable at this precision
pthread	16	none	0.033	23.2	baseline
pthread	16	compact	0.032	13.5	not separable at this precision
pthread	16	scatter	0.032	9.5	not separable at this precision
pthread	20	none	0.038	14.1	baseline
pthread	20	compact	0.037	16.9	not separable at this precision
pthread	20	scatter	0.037	26.1	not separable at this precision
jthread	4	none	0.022	14.4	baseline
jthread	4	compact	0.023	13.8	not separable at this precision
jthread	4	scatter	0.023	25.0	not separable at this precision
jthread	8	none	0.030	29.8	baseline
jthread	8	compact	0.028	28.5	not separable at this precision
jthread	8	scatter	0.027	35.7	not separable at this precision
jthread	16	none	0.037	17.9	baseline
jthread	16	compact	0.037	26.7	not separable at this precision
jthread	16	scatter	0.035	33.0	not separable at this precision
jthread	20	none	0.043	30.3	baseline
jthread	20	compact	0.041	47.6	not separable at this precision
jthread	20	scatter	0.039	31.8	not separable at this precision

- Compact pinning on openmp at 20 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 5.0 percent and the wider of the two spreads is 69.1 percent.
- Scatter pinning on openmp at 20 workers is 12896.0 percent slower than none pinning on the same configuration, against a wider spread of 69.1 percent.
- Compact pinning on pthreads at 4 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 2.1 percent and the wider of the two spreads is 16.9 percent.
- Scatter pinning on pthreads at 4 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 3.7 percent and the wider of the two spreads is 16.9 percent.
- Compact pinning on pthreads at 8 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 2.9 percent and the wider of the two spreads is 26.8 percent.
- Scatter pinning on pthreads at 8 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 7.7 percent and the wider of the two spreads is 26.8 percent.
- Compact pinning on pthreads at 16 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 4.8 percent and the wider of the two spreads is 23.2 percent.
- Scatter pinning on pthreads at 16 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 2.9 percent and the wider of the two spreads is 23.2 percent.
- Compact pinning on pthreads at 20 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 3.3 percent and the wider of the two spreads is 16.9 percent.
- Scatter pinning on pthreads at 20 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 3.9 percent and the wider of the two spreads is 26.1 percent.
- Compact pinning on jthread at 4 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 1.0 percent and the wider of the two spreads is 14.4 percent.
- Scatter pinning on jthread at 4 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 1.3 percent and the wider of the two spreads is 25.0 percent.
- Compact pinning on jthread at 8 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 6.7 percent and the wider of the two spreads is 29.8 percent.
- Scatter pinning on jthread at 8 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 7.5 percent and the wider of the two spreads is 35.7 percent.

- Compact pinning on jthread at 16 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 1.6 percent and the wider of the two spreads is 26.7 percent.
- Scatter pinning on jthread at 16 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 5.4 percent and the wider of the two spreads is 33.0 percent.
- Compact pinning on jthread at 20 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 3.6 percent and the wider of the two spreads is 47.6 percent.
- Scatter pinning on jthread at 20 workers against none pinning on the same configuration is not separable at this precision: the medians differ by 7.6 percent and the wider of the two spreads is 31.8 percent.

Under a hypervisor a binding request constrains guest placement and not silicon, so the expected effect is smaller than the same experiment would show on bare metal. The measurement is reported for what it is, policy by policy, and most of those comparisons are not separable at this precision.

5.6 The cost of determinism

Section 3.2 argued that reproducible reductions are worth having. This is what they cost.

Table 5.7: The price of reproducibility. The deterministic mode combines partial sums in a fixed order that does not depend on the worker count, which is what lets the equivalence suite assert bit identical results; the native mode uses the model’s own reduction. Conjugate gradient is used because its two global reductions per iteration make it the method most exposed to the difference. Each spread column is $(\max - \min) / \text{median}$ over that configuration’s repetitions, median of 15 repetitions. The overhead column reads “not separable at this precision” wherever the difference between the two medians is smaller than the larger of the two spreads. This is the table ground rule 7 was written for: at 1.0.0 it reported effects of minus 5.5 to plus 3.1 percent on runs of seventy two milliseconds, against a block whose median spread was 9.3 percent, and the README quoted those effects as what reproducibility is worth.

backend	deterministic (s)	spread	native (s)	spread	overhead (percent)
openmp	0.871	12.7	0.914	99.3	not separable at this precision
pthreads	5.721	31.6	5.806	14.4	not separable at this precision
jthread	6.004	2.8	5.991	4.6	not separable at this precision
mpi	0.801	8.6	0.821	14.5	not separable at this precision

What the deterministic reduction costs, per backend:

- The deterministic reduction on openmp against the model’s native reduction is not separable at this precision: the medians differ by 4.7 percent and the wider of the two spreads is 99.3 percent.
- The deterministic reduction on pthreads against the model’s native reduction is not separable at this precision: the medians differ by 1.5 percent and the wider of the two spreads is 31.6 percent.
- The deterministic reduction on jthread against the model’s native reduction is not separable at this precision: the medians differ by 0.2 percent and the wider of the two spreads is 4.6 percent.

- The deterministic reduction on mpi against the model’s native reduction is not separable at this precision: the medians differ by 2.4 percent and the wider of the two spreads is 14.5 percent.

Conjugate gradient is used because it performs two global reductions per iteration and is therefore the method most exposed to the difference. On a solver with one reduction per sweep the overhead is proportionally smaller.

Read the spread column before the effect column. Release 1.0.0 quoted this block as a cost of between minus 5 and plus 3 percent and offered it as what reproducibility is worth, on runs whose own repetitions moved further than that. The effects here are mostly smaller than the spread of the rows they are computed from, so what this table establishes is an upper bound rather than a price: the deterministic reduction does not cost enough on these backends for this measurement to see it.

5.7 Scheduling

Table 5.8: What a dynamic schedule costs on uniform work. This is the measurement behind leaving work stealing out of the jthread pool: on a balanced stencil sweep the extra bookkeeping buys nothing. Each spread column is (max - min) / median over that configuration’s repetitions, median of 15 repetitions, and the overhead column reads “not separable at this precision” wherever the difference between the two medians is smaller than the larger of the two spreads. A row that reads the phrase is not evidence that a dynamic schedule is free; it is evidence that this block cannot price it, and the argument for leaving work stealing out has to rest on the rows that do separate.

backend	static (s)	spread	dynamic (s)	spread	dynamic overhead (percent)
openmp	0.063	25.7	0.081	25.6	+28.1
pthreads	0.360	10.1	0.372	11.3	not separable at this precision
jthread	0.372	14.2	0.387	13.7	not separable at this precision

What a dynamic schedule costs against a static one, per backend:

- A dynamic schedule on openmp is 28.1 percent slower than a static one on the same backend, against a wider spread of 25.7 percent.
- A dynamic schedule on pthreads against a static one on the same backend is not separable at this precision: the medians differ by 3.5 percent and the wider of the two spreads is 11.3 percent.
- A dynamic schedule on jthread against a static one on the same backend is not separable at this precision: the medians differ by 4.0 percent and the wider of the two spreads is 14.2 percent.

This is the measurement behind leaving work stealing out of the jthread pool. On a uniform stencil sweep a static partition is already balanced, and the extra bookkeeping of a dynamic schedule buys nothing. The evidence is only as strong as the verdicts above make it: where a backend’s dynamic against static difference is smaller than the spread of those two rows, that backend contributes nothing to the argument, and the sentence above rests on the backends where it is larger.

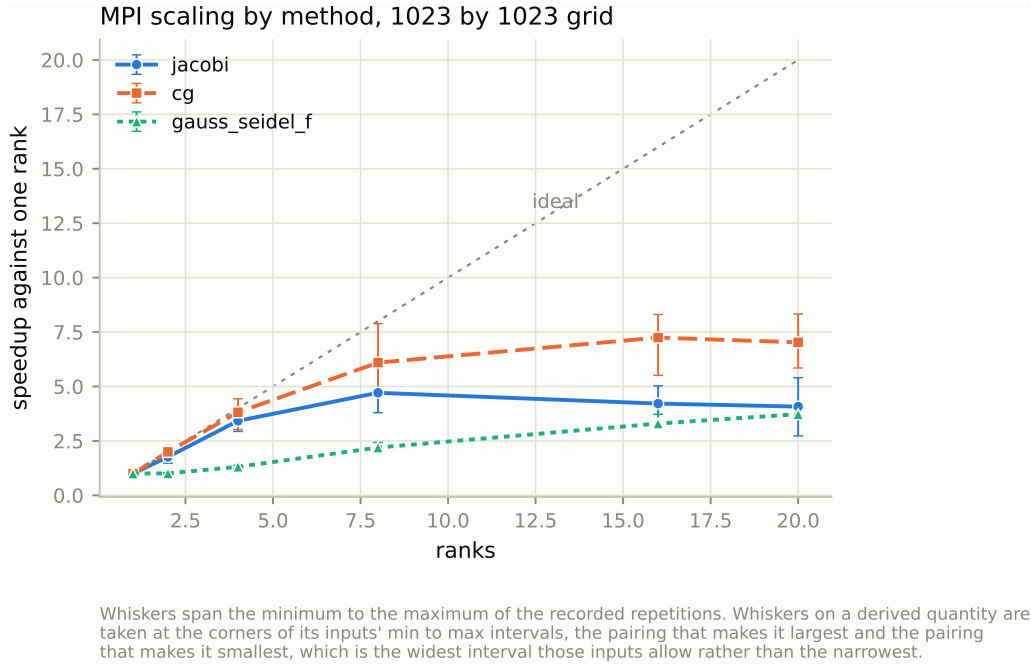


Figure 5.5: Speedup against rank count by method. Jacobi exchanges only halo rows and scales; conjugate gradient additionally performs two global reductions per iteration, each a synchronisation point no amount of local work can hide; natural ordering Gauss Seidel is sequentially dependent and is included to show what that costs when the semantics are preserved rather than quietly replaced.

5.8 Distributed scaling and the communication fraction

The MPI backend records halo, reduction, barrier and ordered pass time separately, so the communication fraction is measured rather than inferred from the gap between observed and ideal speedup.

5.9 The CPU versus GPU comparison

Read this section in the order it is presented. The iteration counts of Section 5.2 come first and are hardware free. The efficiency column below is the comparable one. The seconds column is a property of this particular pair of devices.

5.9.1 Decomposing the speedup

The wall clock ratio between the two devices is not a single fact. It factors into two, and the factorisation is the point:

$$\underbrace{\frac{t_{\text{host}}}{t_{\text{device}}}}_{\text{speedup}} = \underbrace{\frac{B_{\text{device}}}{B_{\text{host}}}}_{\text{memory systems}} \times \underbrace{\frac{\eta_{\text{device}}}{\eta_{\text{host}}}}_{\text{how well each is used}}, \quad (5.1)$$

with B the measured triad bandwidth of each device and η the efficiency of (4.1). The first factor was bought with the graphics card and would be the same for any bandwidth bound kernel. The second is the only part attributable to the implementations.

On the measurements in the table above, the great majority of the observed speedup is the first factor. That is the honest summary: on a bandwidth bound stencil sweep this GPU's measured

Table 5.9: Achieved bandwidth, efficiency against each device’s own measured STREAM triad, and absolute time. The efficiency columns are the comparable ones: they are dimensionless and say how well each device is used. The seconds column is a property of this particular pair of devices and of nothing else. Declared and counted are the two traffic models of Section 4.2, both published because neither has yet been selected. Declared charges the sweeps of each iteration at the conservative byte count, a read for a read and a write for a write. Counted charges the passes over memory at the same count with read for ownership added on the one array a pass writes without reading first, 32 bytes per unknown per pass against the conservative 24. Each percentage divides by the triad counted the same way as its own numerator: the declared column by the plain triad as that triad declares itself, and the counted column by the same triad recounted at 32, since it is a C++ loop whose own store pays the charge the counted model charges the kernel. Counted the same way on both sides the charge cancels, and a counted efficiency is the declared efficiency times the row’s pass count over its sweep count. The byte model therefore moves the achieved bandwidth figure and never the efficiency, which is why the Jacobi rows, one pass to one sweep, carry the same percentage in both columns on both devices. What each column bounds follows from the same arithmetic. The counted model charges every pass a full stencil pass, which is exact for Jacobi and an upper bound for every method whose pass count exceeds its sweep count: a red black sweep is two passes that each write half the unknowns, and conjugate gradient’s six passes are one stencil product, two inner products that write nothing and three vector updates that read what they write. The declared model charges one sweep and is a lower bound for those same methods. Their two columns bracket the traffic rather than compete to describe it, and a counted percentage above one hundred at a size where the sweep streams marks a bound that is loose and not a kernel that outran its memory system. A per pass model that charges each pass what it actually moves is the honest next step and is not in this release. The non temporal triad in the bandwidth table is the instrument that chooses between the two counts, the rule is fixed in benchmarks/sweep_matrix.yaml before the measurement, phase A8b applies that rule to the publication session for release 1.1.0, and phase D4 confirms the outcome with the assembly triad in release 1.2.0 rather than gating it. A streaming efficiency is quoted only where the working set exceeds that device’s last level cache by a factor of four. Rows marked cache resident have a working set smaller than the cache and rows marked partly cached have one within that factor; in both cases a good share of the traffic never reaches memory, so the sweep runs faster than the memory system alone would allow and a percentage would exceed one hundred. That would be the roofline assumption announcing that it does not apply, not a result. The comparison should therefore be read from the largest size, where both devices are unambiguously bandwidth bound. The spread column is (max - min) / median over the repetitions of that row, and both bandwidth columns and both percentage columns move inversely with it: a row whose spread is ten percent has a bandwidth figure good to about ten percent, whichever byte count it is divided by.

method	unknowns	device	working set	GiB/s declared	GiB/s counted	percent d
jacobi	1,046,529	host, 20 threads	24 MiB	367.1	489.4	cache r
jacobi	1,046,529	RTX 5070	24 MiB	850.2	1133.7	cache r
jacobi	4,190,209	host, 20 threads	96 MiB	69.7	93.0	partly
jacobi	4,190,209	RTX 5070	96 MiB	597.1	796.1	partly
jacobi	16,769,025	host, 20 threads	384 MiB	53.3	71.1	
jacobi	16,769,025	RTX 5070	384 MiB	540.2	720.2	
gauss_seidel_rb	1,046,529	host, 20 threads	24 MiB	218.6	583.0	cache r
gauss_seidel_rb	1,046,529	RTX 5070	24 MiB	501.8	1338.3	cache r
gauss_seidel_rb	4,190,209	host, 20 threads	96 MiB	66.1	176.3	partly
gauss_seidel_rb	4,190,209	RTX 5070	96 MiB	369.9	986.4	partly
gauss_seidel_rb	16,769,025	host, 20 threads	384 MiB	30.7	81.9	
gauss_seidel_rb	16,769,025	RTX 5070	384 MiB	272.0	725.3	
sor_rb	1,046,529	host, 20 threads	24 MiB	210.8	562.0	cache r
sor_rb	1,046,529	RTX 5070	24 MiB	492.2	1312.6	cache r
sor_rb	4,190,209	host, 20 threads	96 MiB	67.1	179.1	partly
sor_rb	4,190,209	RTX 5070	96 MiB	368.5	982.8	partly
sor_rb	16,769,025	host, 20 threads	384 MiB	30.6	81.7	
sor_rb	16,769,025	RTX 5070	384 MiB	271.3	723.5	
cg	1,046,529	host, 20 threads	24 MiB	82.2	657.2	cache r
cg	1,046,529	RTX 5070	24 MiB	112.3	898.2	cache r
cg	4,190,209	host, 20 threads	96 MiB	16.4	131.2	partly
cg	4,190,209	RTX 5070	96 MiB	123.6	989.1	partly
cg	16,769,025	host, 20 threads	384 MiB	13.7	109.7	
cg	16,769,025	RTX 5070	384 MiB	110.2	881.7	

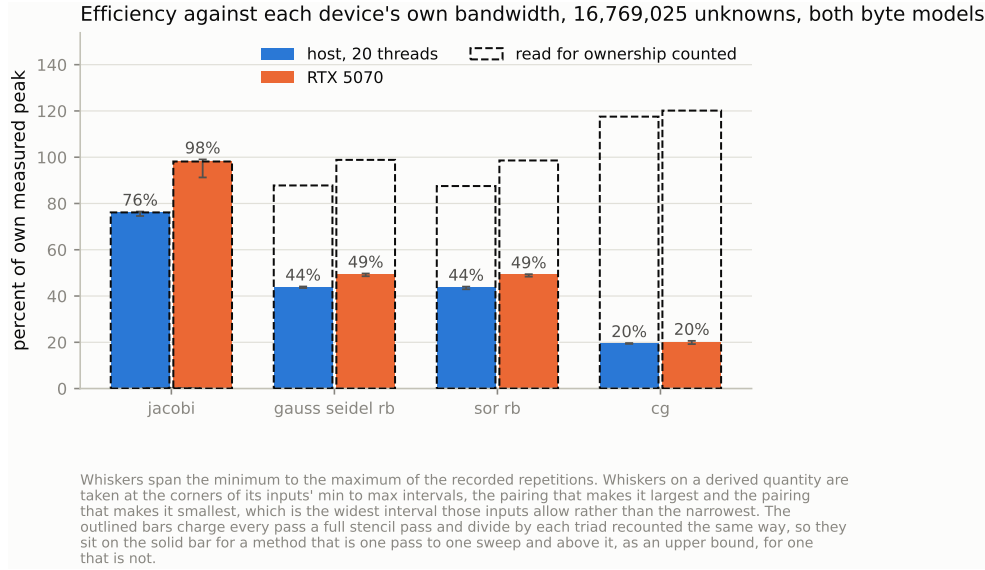


Figure 5.6: Percentage of each device's own measured STREAM triad bandwidth achieved, method by method. This is the fair comparison: it is dimensionless and says how well each device is used, rather than which device is larger. The outlined bars are the counted byte model against each triad recounted the same way, so they sit on the solid bar for a method that is one pass to one sweep and above it, as an upper bound, for one that is not.

triad is 7.9 times the host's, and the device is used somewhat more efficiently while doing it. It is a useful thing to know and considerably less exciting than an undecomposed speedup figure, which is why it is presented this way.

What each column bounds. The two bandwidth columns of Table 5.9 are not two estimates of one quantity. Each percentage divides by the triad counted the same way as its own numerator, so the read for ownership charge cancels in the ratio and a counted efficiency is the declared efficiency times the row's pass count over its sweep count. The byte model therefore moves the achieved bandwidth figure and never the efficiency. Jacobi is one pass to one sweep, so at 16,769,025 unknowns it reads 76.1 percent on the host and 98.1 percent on the device under both models alike.

For a method with more passes than sweeps the counted model charges every pass a full stencil pass, and that overstates the traffic: a red black sweep is two passes that each write half the unknowns, and conjugate gradient's six passes are one stencil product, two inner products that write nothing and three vector updates that read what they write. The counted column is an upper bound for those methods and the declared column a lower one, so the pair brackets the traffic rather than competing to describe it, and conjugate gradient's counted 117.5 percent on the host and 120.2 percent on the device are a bound that is loose rather than a kernel that outran its memory system. A per pass model that charges each pass what it actually moves is the honest next step. It wants the traffic model settled first and is release 1.2.0 work, so this release brackets the traffic instead of estimating it.

Which traffic model the measurement selects, under the rule fixed before it was taken:

- The non temporal triad reached a ratio of 1.0242 against the plain triad on the publication machine, between the 1.10 and 1.20 thresholds fixed before the measurement, so the traffic model is recorded as unresolved: both counts, 24 and 32 bytes per unknown per pass, are carried in every table and figure of this report, neither is presented on its own as the achieved bandwidth, and no claim is made that rests on one of them and would fail under

the other.

- The statistic is the median over 5 repetitions of the two probes at 8 workers, it reads 1.0242, and the interval from the smallest to the largest of those ratios is 0.7844 to 1.1304. The interval 0.7844 to 1.1304 over 5 repetitions contains 1.10, so the statistic does not clear its own spread.

5.9.2 The crossover, derived

Because iteration counts and per iteration costs are measured separately, the point at which one method on one device overtakes another can be computed from (4.2) rather than declared. Both sides of that expression are tabulated in this chapter, so a reader can substitute their own hardware by replacing the bandwidth term with a triad measurement of their own.

Chapter 6

Discussion

6.1 What the equivalence invariant bought

The decision that shaped everything else was to fix the reduction chunk grid by problem size alone. It looked at first like a small implementation detail and it turned out to be the foundation of the whole comparison.

Because every shared memory backend produces bit identical iterates, the timing differences between them cannot be attributed to any numerical difference: there is none. That removes an entire class of objection from the results chapter. It also turned the test suite into something sharper than a regression check. Asserting exact equality means a test fails on the first bit that differs, which is a much more sensitive detector than a tolerance that would have absorbed real faults silently. The fused multiply add discrepancy in Section 3.5 was one bit in the last place, and a tolerance based test would never have reported it.

The cost is real and is measured in Section 5.6 rather than excused. It is worth paying here because reproducibility is a deliverable of this project; a production solver with different priorities might reasonably choose the native reduction and say so.

6.2 The methods that do not parallelise, and why they are still here

Natural ordering Gauss Seidel, SOR and block Gauss Seidel are sequentially dependent. There was a real temptation to substitute a red black reordering whenever more than one worker was present, which would have produced attractive scaling curves for all of them.

That would have been a lie of a particular kind: not a wrong number, but a right number for a different algorithm. The library instead preserves exact sequential semantics on every backend, including across MPI ranks through a pipelined token chain. The consequence is that these methods appear in the scaling figures as essentially flat lines, and that is the correct result. The reader learns something true, which is that a method with a sequential recurrence has nothing to offer a parallel machine, and that red black ordering is the price one pays to get parallelism back.

Charging that price explicitly also improved the device comparison. Because the red black penalty is measured (80,906 against 78,516 iterations on a 255 grid, 3.0 percent), it can be applied to both the CPU and the GPU side, and the device figure is not silently inflated by a change of algorithm.

6.3 Where the environment set the limits

Three findings came from the environment rather than from the mathematics, and all three changed what could be claimed.

The hybrid core topology is invisible from inside WSL2. The guest synthesises a homogeneous machine, and thread affinity binds to a virtual processor the hypervisor may place anywhere. The response was to measure rather than assume, and to take the knee from the aggregate scaling curve, which does not depend on identifying individual cores. That is a weaker claim than the objective originally envisaged and it is the strongest one the environment supports.

The CUDA toolchain and the host toolchain cannot be the same. CUDA 13.3 rejects any host compiler newer than GCC 15 and cannot parse GCC 15's own headers either, so no released GCC serves as both. Resolving this through a C ABI boundary was not a workaround so much as a clarification: the boundary was already required by the rule that backends do not leak their model's types, and enforcing it at the ABI level made that rule impossible to violate by accident.

MPI version macros describe conformance, not capability. OpenMPI 5.0.10 advertises MPI 3.1 while implementing many MPI 4.0 entry points. Guarding on the version macro would have disabled working functionality; assuming the functions were present would have broken elsewhere. Probing for symbols is the only approach that is both correct and portable.

6.4 Things that were wrong before they were right

Four faults are worth naming because each was the kind that produces a plausible number rather than an obvious failure.

The manufactured solution was an eigenvector. With $u = \sin(\pi x) \sin(\pi y)$ the Krylov subspace is one dimensional and conjugate gradient converges in one iteration at every grid size. That looks like a spectacular result and is a degenerate problem. It was caught because a test asserted the iteration count should *grow* with the grid, not merely that the method converged. A test that had only checked convergence would have passed forever.

A default that was right for one solver and wrong for another. The relaxation factor defaulted to one, which is harmless for SOR, where it means Gauss Seidel, and catastrophic for Richardson, where on the Poisson operator it gives an iteration matrix with spectral radius seven. The lesson generalises: a shared options struct wants defaults that mean "ask the component", not defaults that happen to suit whichever component was written first.

The resumable sweep was not resumable. The skip test sat after the work rather than before it, so a restart re-ran every configuration and skipped only the write. The counters reported plausible numbers throughout, which is why it survived until a restart was timed rather than merely observed.

A tolerance below the arithmetic. Two tests asked for a relative residual that stationary iterations on this operator cannot reach, because over relaxation raises their rounding limited floor above it. The failure appeared only when the solver those tests used changed, which made it look like a regression in the solver rather than an impossible request. Measuring the floor turned a confusing symptom into a table, and the tests now ask for a tolerance above it.

6.5 What the traffic model changes, and what it does not

The open question of Section 4.2.4 is whether a Jacobi pass over the five point stencil moves 24 bytes per unknown or 32, the extra eight being the output line an ordinary store has to fetch before it can overwrite it. It is worth being exact about what turns on the answer, because less turns on it than an earlier draft of this section claimed.

An efficiency is one achieved bandwidth over another, and it says something only when both sides are counted the same way. The triad this report divides by is a plain C++ loop, so its own store pays exactly the charge the counted model charges the kernel, and under the counted model the denominator is recounted too. The charge then cancels, and a counted efficiency is the declared efficiency times the row's pass count over its sweep count. The byte model moves the achieved bandwidth figure in GiB/s and leaves the efficiency where it was, on the host and on the device alike. Host Jacobi stands at 76.1 percent of the plain triad and the device at 98.1 percent of its own, under either model, and the two percentage columns of Table 5.9 agree on those rows to the digit they print.

The Jacobi row is therefore not an exception waiting for a byte model to remove it, and it is not an exception at all. Its device over host efficiency ratio is 1.29 in this generation, where release 1.0.0's published generation put it far higher; the before and after per phase is in `PROGRESS.md` rather than typed here. What moved it is not the traffic model. It is that the per iteration full state copy is gone from the Jacobi path and that the timed region no longer holds the allocation, so the host figure is a measurement of the sweep rather than of the sweep plus a serial copy plus a workspace. That is the correction that mattered, and it is a correction to the measurement and not to the model that reads it.

The other three kernels gave that conclusion independently and Jacobi now joins them. At 16,769,025 unknowns the device over host efficiency ratio is 1.13 for red black Gauss Seidel and 1.02 for conjugate gradient, against Jacobi's 1.29, so on these kernels the great majority of the observed speedup is the memory system and little of it is attributable to the port. That holds under the conservative count, without the correction the traffic model question was about, which is a better outcome than needing it.

The sentence this replaces. An earlier draft of this section said that under read for ownership the host achieved bandwidth and the host efficiency both rise by a third while the device figures do not move, and that host and device efficiency converge on the Jacobi row as a result. The first half is right about GiB/s and wrong about the efficiency, and the rest was the same mismatch: it divided a numerator counted one way by a denominator counted the other. The pre registered sentence quoted in Section 4.2.4 carries the same reasoning, and it is quoted there exactly as it was written, because a pre registration edited after the measurement is not one. This paragraph is the correction to it, and the outcome that sentence was written for is not the one the measurement selected.

The report does not choose between the two models on its own authority. The rule was fixed before the measurement, the measurement is the publication session's, and the generator applies the rule and writes the sentence, which Section 5.9 carries. That sentence records the outcome

as unresolved, which is this instrument saying it cannot separate the two counts on this machine rather than choosing between them.

Both counts are therefore carried in every table and every figure, every claim in this chapter that would change under the other model is stated conditionally, and no conclusion here rests on one of them in a way that would fail under the other. For a method whose pass count exceeds its sweep count the two columns bracket the traffic rather than compete to describe it, which Section 5.9 sets out; for Jacobi they are the same number.

6.6 Related work

Deterministic reduction is established technique and nothing here invented it. Saying so is not modesty: the contribution of this project is a narrower thing than the reduction, and not naming the prior art both reads as unawareness of the field and undersells the work.

Reproducible arithmetic. Intel’s Math Kernel Library offers Conditional Numerical Reproducibility [17], which gives bitwise identical results across runs and across a range of processors subject to conditions the vendor documents: a fixed thread count, aligned arrays, and a selected code path. It states the shape of the problem clearly, which is that reproducibility is bought by fixing what a performance library would otherwise be free to vary. Demmel and Nguyen give summation algorithms whose result does not depend on the order or the number of summands, at bounded cost and with an error bound [9, 10], and Ahrens, Demmel and Nguyen develop the binning algorithm behind ReproBLAS [1]. That is a stronger guarantee than the one made here: ReproBLAS is reproducible across different decompositions, whereas the fixed chunk grid of this library is reproducible because the decomposition is held fixed.

Performance portability. Kokkos [12, 25] and RAJA [5] are the established C++ layers for writing a kernel once and running it over several execution models including GPUs. Both provide reductions and both document what is and is not deterministic about them. They solve the portability problem this library also solves, at a scale and maturity this library does not approach.

Solver libraries. Ginkgo [2] is a modern C++ sparse linear algebra framework built around linear operators, with GPU backends and a strong benchmarking culture. PETSc [4, 3] is the reference distributed solver toolkit, whose Krylov methods and preconditioners this repository implements a small subset of. hypre [13] is the reference library for scalable multigrid preconditioners. Each is a real answer to the question of what to use to solve a system, and this library is not.

What this project adds. Not the deterministic reduction. Three things, and they are narrower.

1. **One numerical invariant held across seven execution models at once**, a GPU and a distributed backend among them. The comparison between programming models in Chapter 5 is only meaningful because the thing being executed is provably identical, and holding one contract across that particular set of models is the part that is unusual.
2. **Asserted as exact equality in a test suite**, rather than described in documentation. The equivalence suite compares with `==` across twelve solvers, two problem families, four

backends and seven worker counts, and fails on the first bit that differs. The one bit contraction discrepancy of Section 3.5 was found that way and would have vanished into any tolerance.

3. **Priced.** What the invariant costs is measured against each model's own native reduction and reported with its run to run spread, including where that spread is wide enough that the honest answer is that the cost cannot be separated from the noise.

What is missing, and when it arrives. There is no baseline comparison in this report. Nothing here says what a competent existing implementation achieves on this machine on this problem, so a statement of the form "this solver reaches some percentage of host STREAM" cannot be read as good or bad. That is a real gap and it is stated as one. A single baseline against PETSc is planned for release 1.2.0: the same stencil, KSPCG with PCJACOBI and with PCSOR, the same tolerance, the same machine and the same reported metrics, emitting rows in the schema the sweep already uses. One baseline is enough, and the result is worth publishing whichever way it comes out. `docs/RELATED_WORK.md` carries this section in longer form.

6.7 What a reader should take away

For someone choosing a programming model, the results chapter says what each model costs on identical work, with the numerical variable removed by construction. For someone choosing a method, the convergence tables say what each one costs in iterations, independent of any machine. For someone weighing a GPU, Section 5.9 separates the part of the speedup that is the memory system from the part that is the implementation, which is the separation that determines whether the speedup would transfer to their problem.

The most transferable conclusion is methodological rather than numerical. A comparison between execution models is only meaningful if the thing being executed is identical, and the only way to know that is to test it at the bit level. Everything else in this project follows from taking that seriously.

Chapter 7

Conclusion

7.1 What was built

A C++20 numerical library in which twelve iterative solvers are written once, against one problem interface, and run unchanged over seven execution backends. Around them: a numerics module with root finders, quadrature rules, ODE integrators and dense factorisations, each with its convergence order tested against theory; two problem families; a benchmark harness that resolves a declared matrix, resumes after interruption and records the provenance of every row; and two reports built from the measurements by script.

7.2 What was established

1. **Identical numerics across execution models is achievable, not merely approachable.** With deterministic reductions every shared memory backend at every worker count produces bit identical iterates, and the GPU sweeps match the CPU bit for bit as well. This is asserted by the test suite as exact equality across twelve solvers, two problem families, four backends and seven worker counts, and it required fixing the reduction chunk grid by problem size and disabling fused multiply add contraction on both host and device.
2. **The classical convergence theory holds to the digit.** The Jacobi spectral radius matches $\cos(\pi h)$; Gauss Seidel needs exactly half the iterations of Jacobi, as $\rho_{GS} = \rho_J^2$ requires; Young's closed form relaxation factor is a genuine minimum; optimal relaxation changes the growth order from $O(n^2)$ to $O(n)$; and conjugate gradient reaches the same order without being told a parameter. The five point stencil is second order accurate with the error constant $\pi^2/12$ predicted by the leading truncation term.
3. **Sequential methods do not parallelise, and saying so is more useful than hiding it.** Natural ordering Gauss Seidel keeps exact semantics on every backend, including across MPI ranks, and shows essentially no speedup. Red black ordering is what buys the parallelism, at a measured cost of 3.0 percent more iterations on a 255 grid.
4. **A GPU speedup on a bandwidth bound kernel is mostly the memory system.** Decomposed through (5.1), the observed advantage separates into a large factor that is the difference in measured bandwidth and a small factor that is the difference in how well each device is used. Reporting the undecomposed ratio would have been accurate and misleading.

7.3 What was not established

Stated as plainly as the conclusions, because a limitation left implicit reads as a claim.

- **The performance versus efficiency core split could not be identified from inside the guest.** WSL2 synthesises a homogeneous topology and affinity binds to a virtual processor. The knee is reported from the aggregate scaling curve instead, which is a weaker claim than originally intended and the strongest the environment supports.
- **Nothing is established about compute bound problems.** Every conclusion about devices here concerns bandwidth bound stencil work. A dense factorisation would rank the hardware differently.
- **Nothing is established about single solve latency,** power, cost, or programmer effort. Transfer time is reported separately rather than folded in; the rest were not measured and are not claimed.
- **The iteration counts at the largest grids are extrapolated,** not measured, for the methods whose counts grow like n^2 . The extrapolation rests on closed form rates that were verified at the sizes where measurement was affordable, and the report says which numbers are which.

7.4 Further work

Preconditioned conjugate gradient. The symmetric splittings are already implemented and symmetric Gauss Seidel and SSOR are exactly the preconditioners the method wants. Combining them would be a small amount of code and would demonstrate the point of the symmetric variants, which currently exist in the zoo without the application that motivates them.

Multigrid. Every component is present: smoothers in the relaxation sweeps, an exact coarse solve in the LU routine, and the grid structure in the Poisson problem. Multigrid is the method that makes the iteration count independent of the grid entirely, and adding it would put the $O(n^2)$ and $O(n)$ results of this report in their proper context as the two things it improves on.

A local slab MPI decomposition. Each rank currently allocates the whole grid. The communication volume is already correct, so this would not change any timing reported here, but it would remove the memory ceiling that keeps the distributed runs below the sizes the device comparison uses.

Bare metal measurement of the core knee. The one objective the environment prevented. Running the same sweep outside a hypervisor would settle whether the knee in the aggregate curve coincides with the performance core count, which is currently inferred rather than confirmed.

Mixed precision. The stencil sweeps are bandwidth bound, so halving the width of the iterate would come close to halving the time. Whether the resulting convergence penalty is worth it is exactly the kind of question this library's separation of iteration count from per iteration cost is built to answer.

Bibliography

- [1] Willow Ahrens, James Demmel, and Hong Diep Nguyen. Algorithms for efficient reproducible floating point summation. *ACM Transactions on Mathematical Software*, 46(3):1–49, 2020. Article 22. The binning algorithm this paper describes is the one implemented by ReproBLAS.
- [2] Hartwig Anzt, Terry Cojean, Goran Flegar, Fritz Göbel, Thomas Grützmacher, Pratik Nayak, Tobias Ribizel, Yuhsiang Mike Tsai, and Enrique S. Quintana-Ortí. Ginkgo: a modern linear operator algebra framework for high performance computing. *ACM Transactions on Mathematical Software*, 48(1):1–33, 2022. Article 2.
- [3] Satish Balay, Shrirang Abhyankar, Mark F. Adams, et al. PETSc/TAO users manual. Technical Report ANL-21/39, Revision 3.25, Argonne National Laboratory, 2026. Cited alongside the 1997 paper because the manual is where the current solver and preconditioner interfaces are documented, and the baseline comparison planned for release 1.2.0 is written against them. Available at <https://petsc.org/release/manual/>.
- [4] Satish Balay, William D. Gropp, Lois Curfman McInnes, and Barry F. Smith. Efficient management of parallelism in object oriented numerical software libraries. In Erlend Arge, Are Magnus Bruaset, and Hans Petter Langtangen, editors, *Modern Software Tools for Scientific Computing*, pages 163–202. Birkhäuser, Boston, 1997.
- [5] David A. Beckingsale, Thomas R. W. Scogland, Jason Burmark, Richard Hornung, Holger Jones, William Killian, Adam J. Kunen, Olga Pearce, Peter Robinson, and Brian S. Ryujin. RAJA: portable performance for large scale scientific applications. In *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, pages 71–81, 2019.
- [6] Richard P. Brent. *Algorithms for Minimization without Derivatives*. Prentice Hall, Englewood Cliffs, 1973.
- [7] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. *Numerical Analysis*. Cengage Learning, Boston, 10 edition, 2016.
- [8] Philip J. Davis and Philip Rabinowitz. *Methods of Numerical Integration*. Academic Press, Orlando, 2 edition, 1984.
- [9] James Demmel and Hong Diep Nguyen. Fast reproducible floating point summation. In *2013 IEEE 21st Symposium on Computer Arithmetic*, pages 163–172, 2013.
- [10] James Demmel and Hong Diep Nguyen. Parallel reproducible summation. *IEEE Transactions on Computers*, 64(7):2060–2070, 2015.
- [11] J. R. Dormand and P. J. Prince. A family of embedded runge kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

- [12] H. Carter Edwards, Christian R. Trott, and Daniel Sunderland. Kokkos: enabling manycore performance portability through polymorphic memory access patterns. *Journal of Parallel and Distributed Computing*, 74(12):3202–3216, 2014.
- [13] Robert D. Falgout and Ulrike Meier Yang. hypre: a library of high performance preconditioners. In *Computational Science, ICCS 2002*, volume 2331 of *Lecture Notes in Computer Science*, pages 632–641, Berlin, 2002. Springer.
- [14] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, 4 edition, 2013.
- [15] Georg Hager and Gerhard Wellein. *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, Boca Raton, 2010.
- [16] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, Berlin, 2 edition, 1993.
- [17] Intel Corporation. *Developer Guide for Intel oneAPI Math Kernel Library for Linux: Obtaining Numerically Reproducible Results*. Intel Corporation, 2025. Cited as vendor documentation because Conditional Numerical Reproducibility is a product feature with no paper of its own. Available at <https://www.intel.com/content/www/us/en/docs/onemkl/developer-guide-linux/>.
- [18] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE Computer Society Technical Committee on Computer Architecture Newsletter*, pages 19–25, 1995.
- [19] Michael McCool, Arch D. Robison, and James Reinders. *Structured Parallel Programming: Patterns for Efficient Computation*. Morgan Kaufmann, Waltham, 2012.
- [20] Message Passing Interface Forum. *MPI: A Message Passing Interface Standard, Version 4.0*. University of Tennessee, Knoxville, 6 2021.
- [21] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 6.0*. OpenMP Architecture Review Board, 11 2024.
- [22] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, Philadelphia, 2 edition, 2003.
- [23] Jonathan Richard Shewchuk. An introduction to the conjugate gradient method without the agonizing pain. Technical Report CMU-CS-94-125, School of Computer Science, Carnegie Mellon University, Pittsburgh, 1994.
- [24] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, 1997.
- [25] Christian R. Trott, Damien Lebrun-Grandié, Daniel Arndt, Jan Ciesko, Vinh Dang, Nathan Ellingwood, et al. Kokkos 3: programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):805–817, 2022.
- [26] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [27] David M. Young. *Iterative Solution of Large Linear Systems*. Academic Press, New York, 1971.